The building blocks of deep learning models - part 1

A light overview

Filippo Biscarini
Senior Scientist
CNR, Milan (Italy)

Nelson Nazzicari
Research fellow
CREA, Lodi (Italy)



Deep learning: the building blocks

1. Function approximation
2. The neural network model:
   a. the “neuron”
   b. the network
3. Activation functions

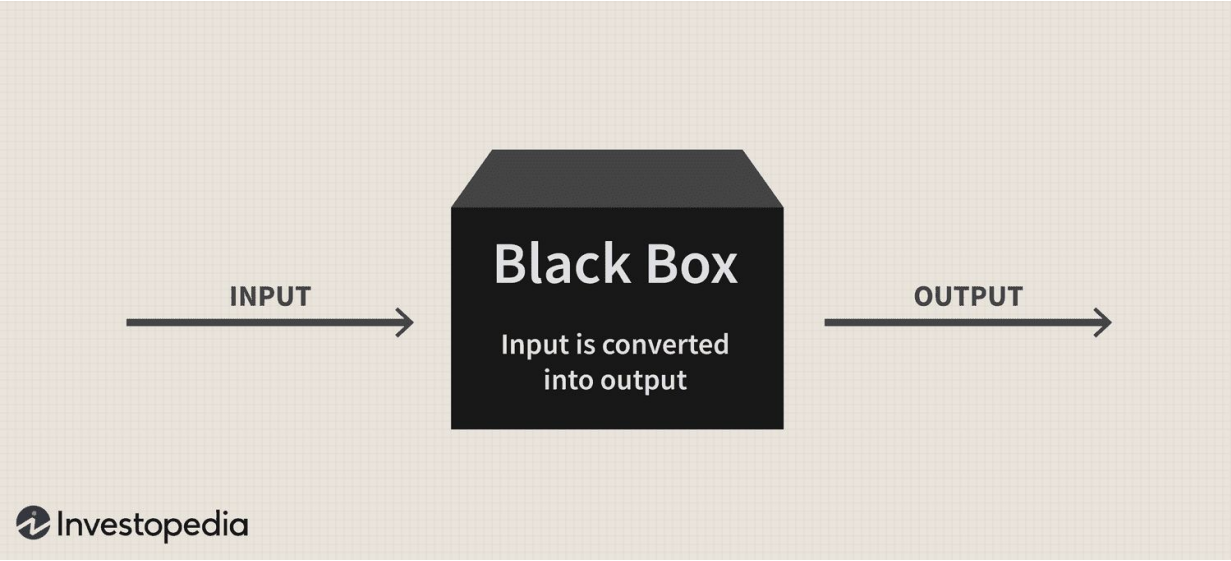
This session

4. Cost functions
5. Gradient descent (and solvers/optimizers)
6. Forward propagation and the backward propagation algorithm

Through the logistic regression example



Let’s crack the black box open



]From: https://www.investopedia.com/terms/b/blackbox.asp]

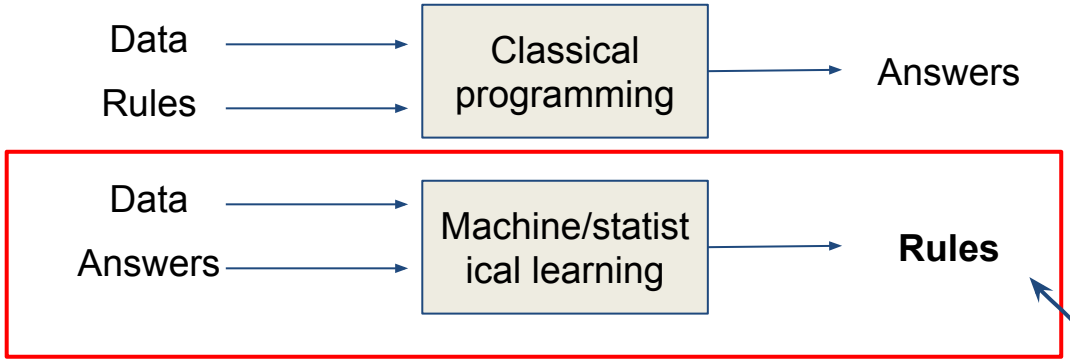
- NN models are said to be “black boxes”
- Let’s see what’s inside the box!



Function approximation



Function approximation?

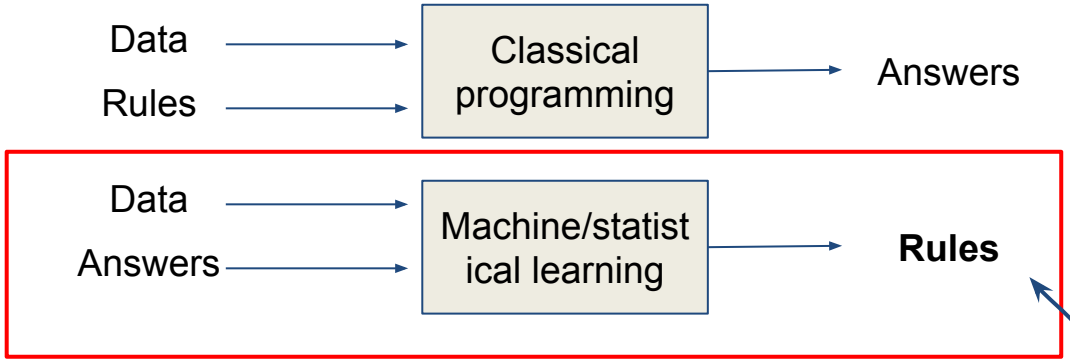


- **unknown function** that maps input data to output results (answers):

  » y = f(x) ← **[mapping function]**



Function approximation?



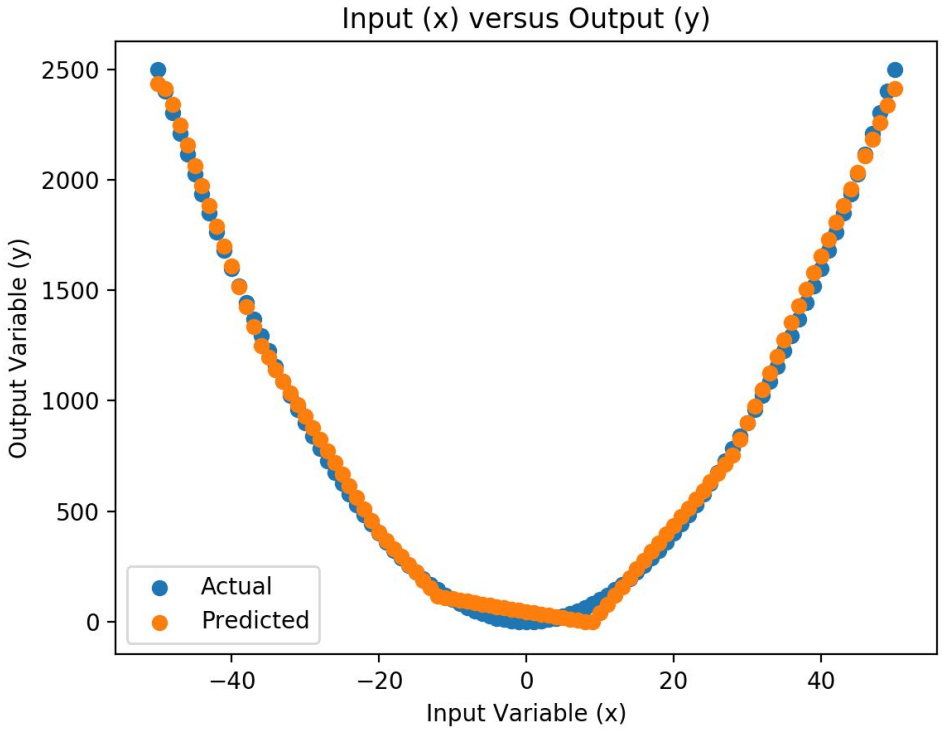
- **unknown function** that maps input data to output results (answers):

  » y = f(x) ← **[mapping function]**

- learn this function → **function approximation**
- f(x) can be **nonlinear** and quite **complex**



Function approximation - intuition



- (known) **quadratic function** (blue line)
- approximated with a **neural network model** [2 hidden layers with 10 nodes each] (orange line)

[From: https://machinelearningmastery.com/neural-networks-are-function-approximators/]



The universal approximation theorem

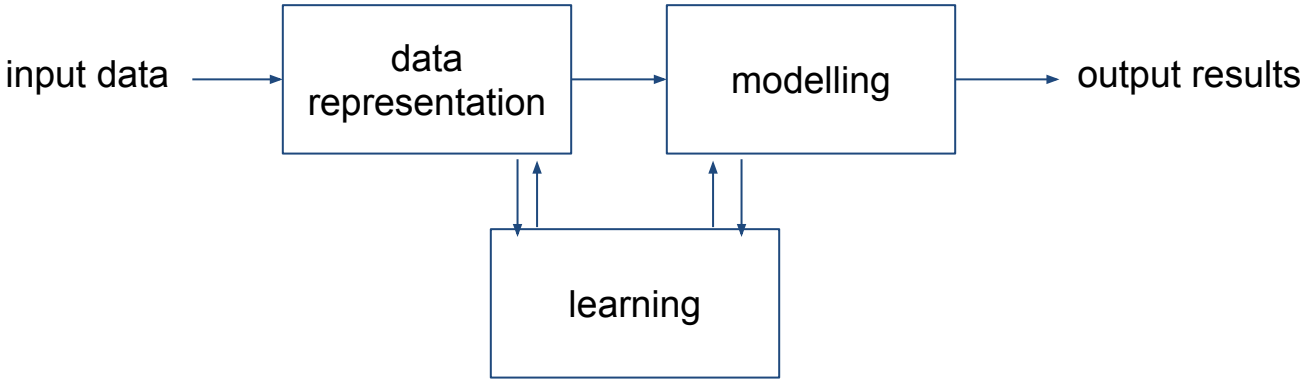
neural networks can approximate **any function**: "*no matter what the function, there is guaranteed to be a neural network so that for every possible input, x, the value f(x) (or some close approximation) is output from the network*"

- naming a piece of music based on a short sample of the piece
- predicting a future phenotype (e.g. disease risk) from genomic data
- translating an Italian text into English (many possible functions, since there are often many acceptable translations of a given text)

powerful learning algorithms + universality → success of DL!



Function approximation



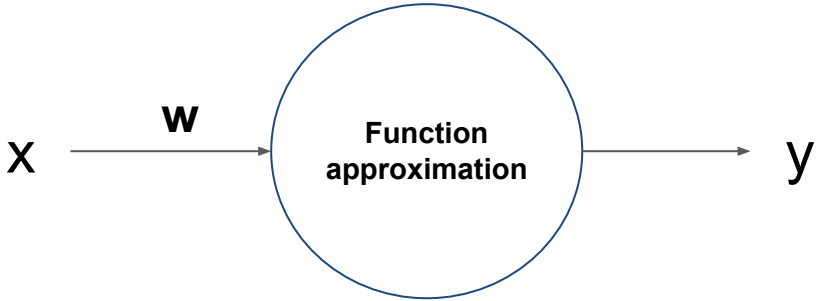
- NNs are ~~good~~excellent at finding functions that accurately map x to y
- deep neural networks (NNs) are powerful **function approximators**
  - » y = f(x)
- ¡complex highly non-linear functions can lead to problems with **generalization!**



The neural network model: anatomy of a neural network



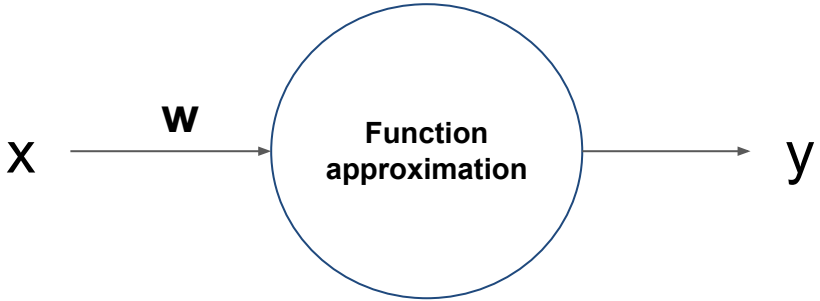
Neural network, the basic unit: the “neuron”



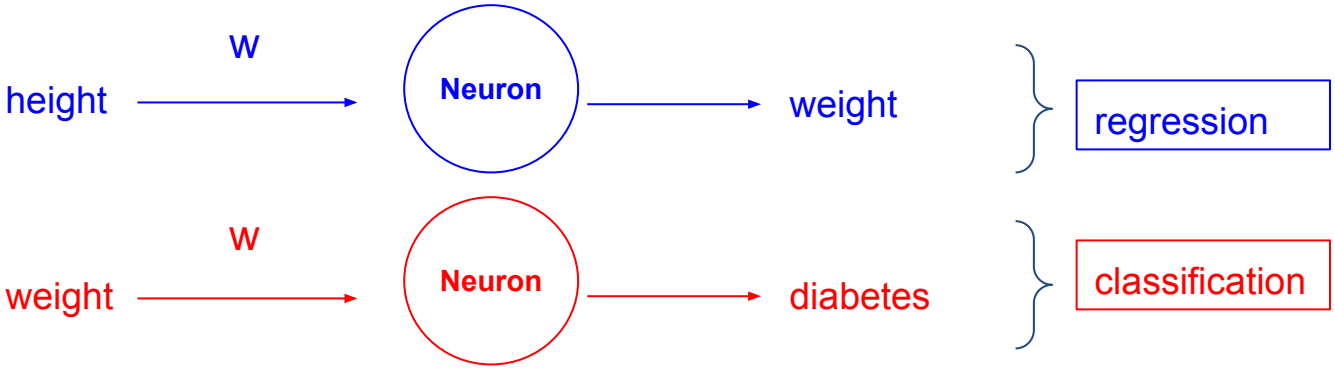
- Mc Culloch & Pitts (1943)
- **perceptron** (“neuron”):
  - dendrites
  - neuron
  - axon



Neural network, the basic unit: the “neuron”

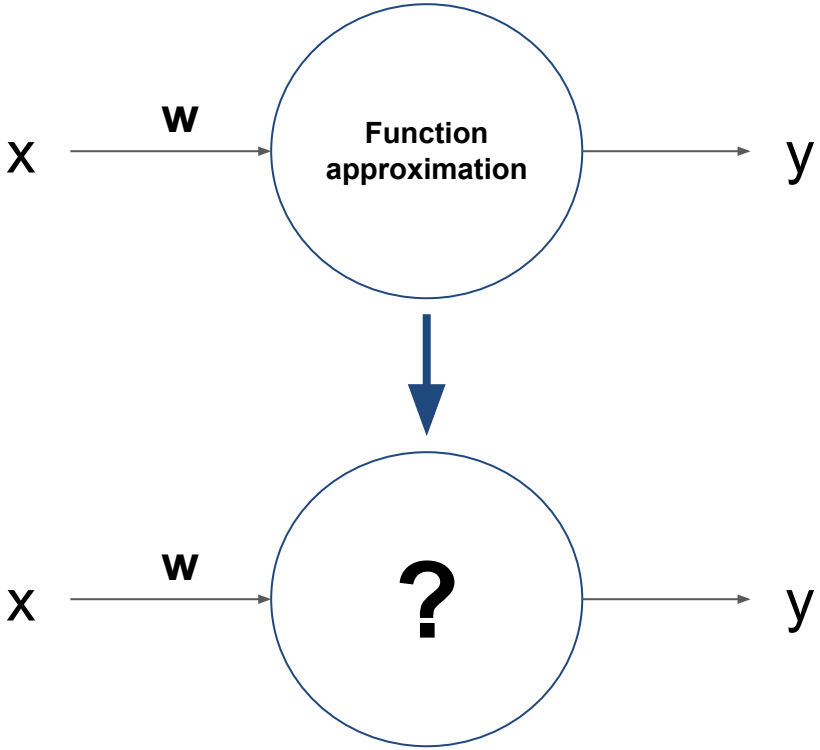


- Mc Culloch & Pitts (1943)
- **perceptron** (“neuron”):
  - dendrites
  - neuron
  - axon





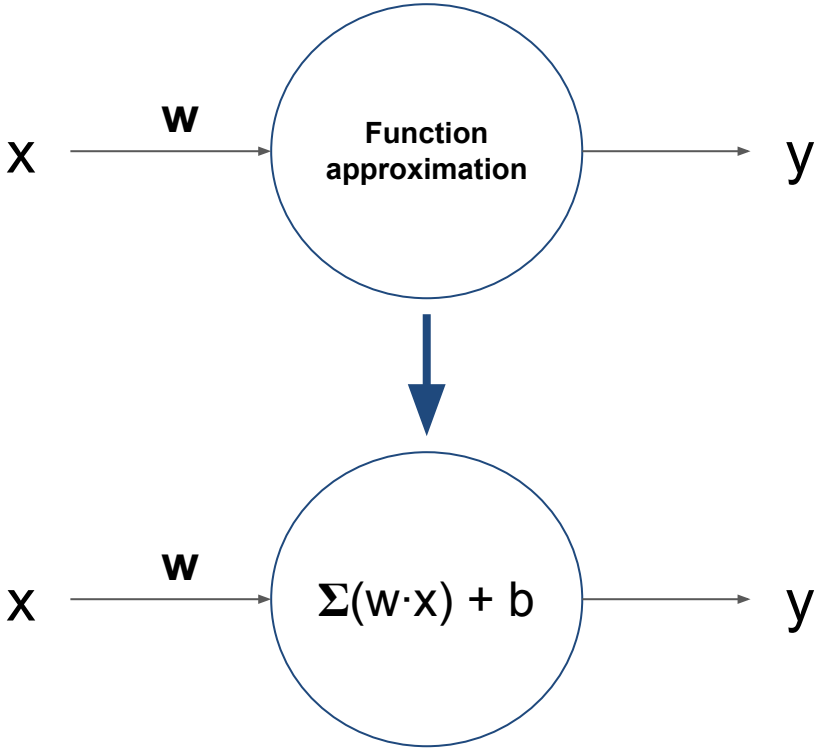
Neural network, the basic unit: the “neuron”



- Mc Culloch & Pitts (1943)
- **perceptron** (“neuron”):
  - dendrites
  - neuron
  - axon



Neural network, the basic unit: the “neuron”

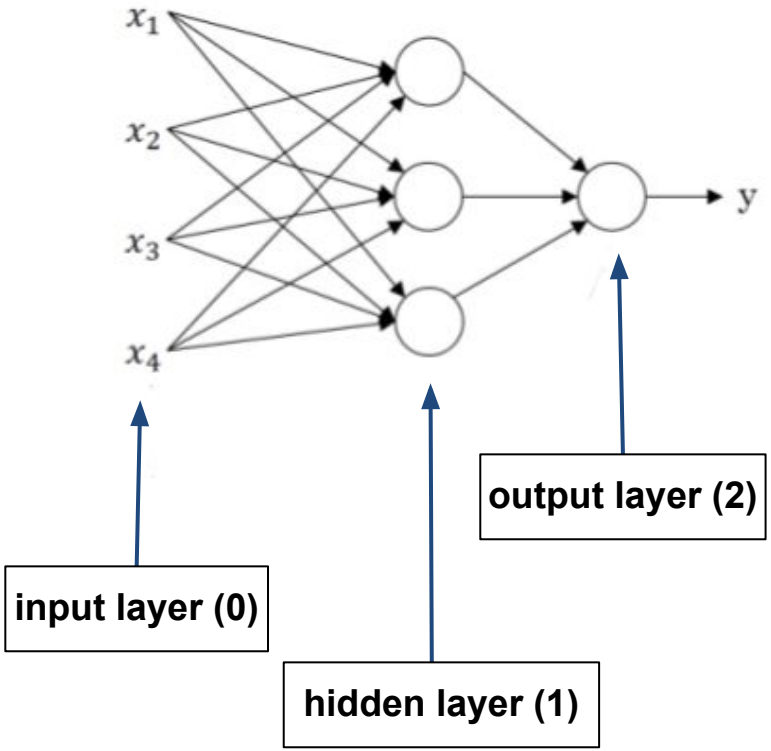


- Mc Culloch & Pitts (1943)
- **perceptron** (“neuron”):
  - dendrites
  - neuron
  - axon
- **learning the weights**

- e.g. linear combination of weights*features + bias
- fancy way to perform linear regression
- solved through NN rather than OLS or ML



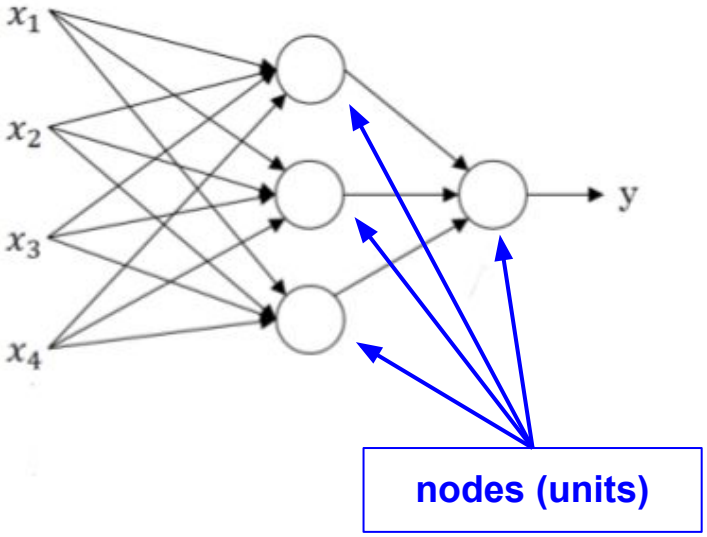
Fully connected (dense) neural network



- two-layer NN (not strictly “deep”):
  - input layer: [0]
  - hidden layer: [1]
  - output layer: [2]



Fully connected (dense) neural network

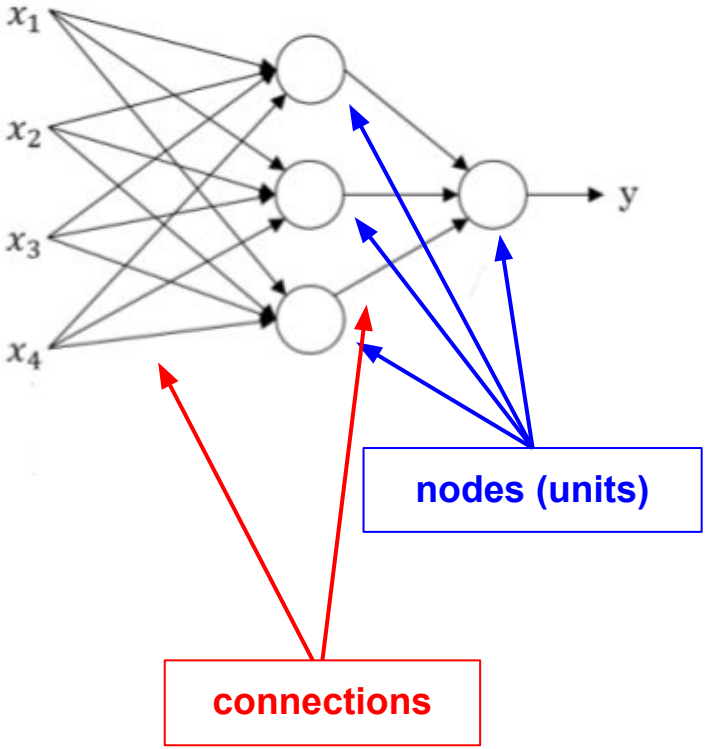


IMPORTANT!

- each hidden unit takes in input all x features
- replicates the predictive model as many times as there are units (“neurons”)
- if the approximated function is linear regression, each unit will fit a different linear regression model
  - e.g.: 3 units → 3 regression models



Fully connected (dense) neural network



- **two-layer NN** (not strictly "deep"):
  - input layer: [0]
  - hidden layer: [1]
  - output layer: [2]
- all features connected to all "neurons" in the hidden layer
- the NN will decide which variables to use (and how) in each node (by **learning the weights**)

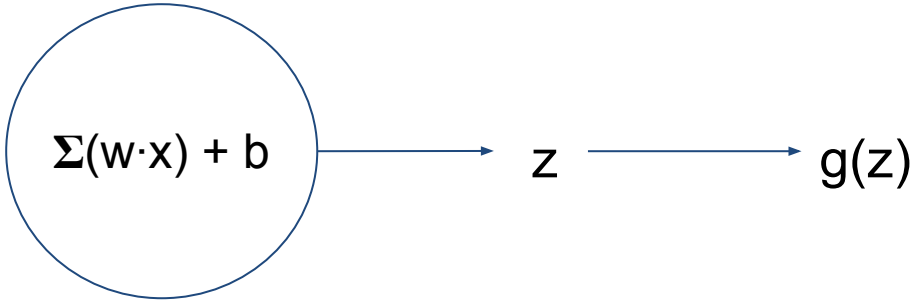


Activation functions



Activation functions: what?

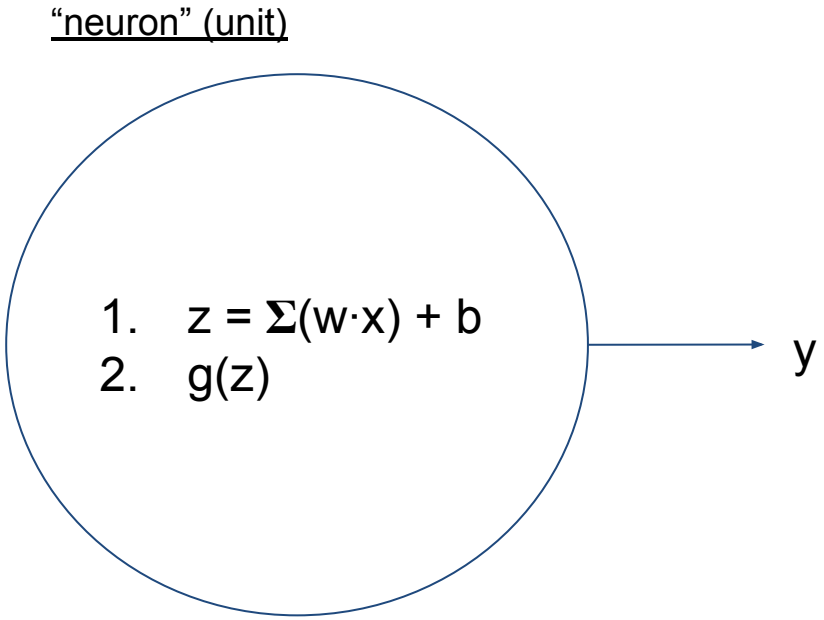
“neuron” (unit)



- g(z): **activation function**



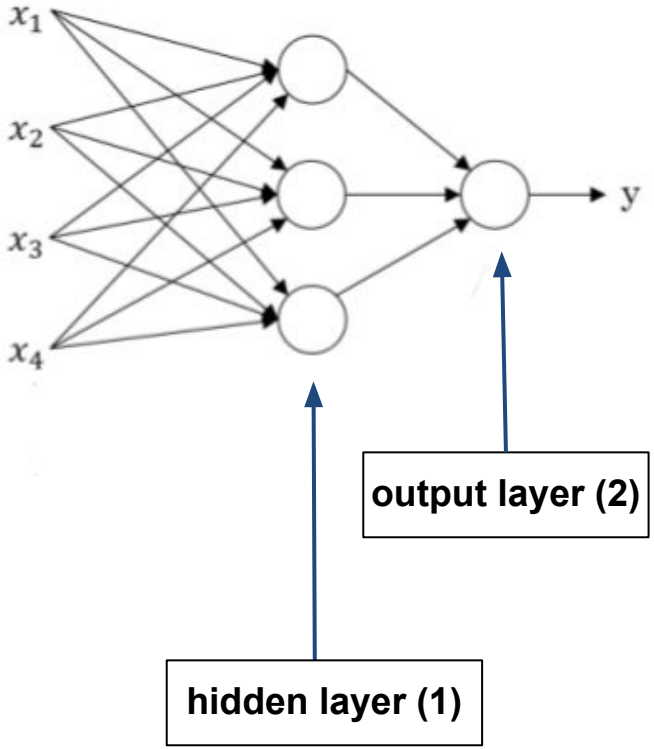
Activation functions: what?



- g(z): **activation function**
- the unit actually processes both the combination of weights and features and the activation function
- the output can be i) the final prediction, or ii) the intermediate output of a hidden layer



Activation functions: when and where?



- **when**: each time a unit is activated: input data (initial features, intermediate output) is processed and output is transferred to the next layer (or final output) through an activation function
- **where**: hidden layers and output layer

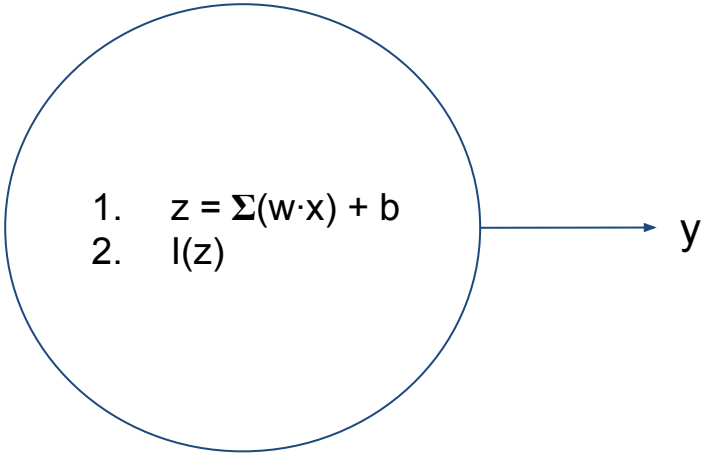


Activation functions: which one?

- Identity function
- Logistic function (sigmoid)
- Hyperbolic tangent function (tanh)
- ReLU (Rectified Linear Unit) function
- Softmax function
- … and many more



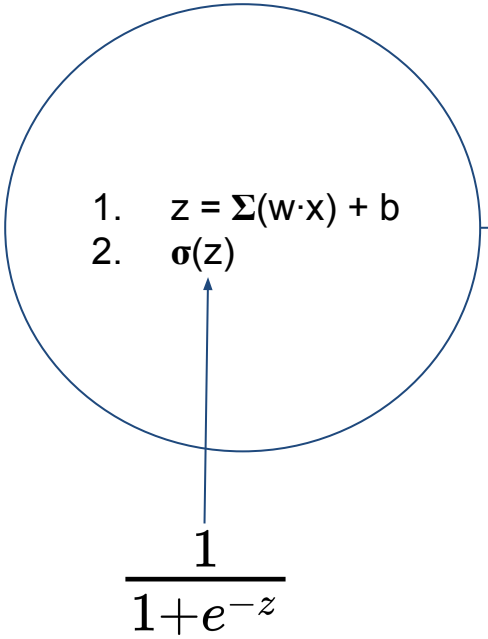
Activation functions: identity function



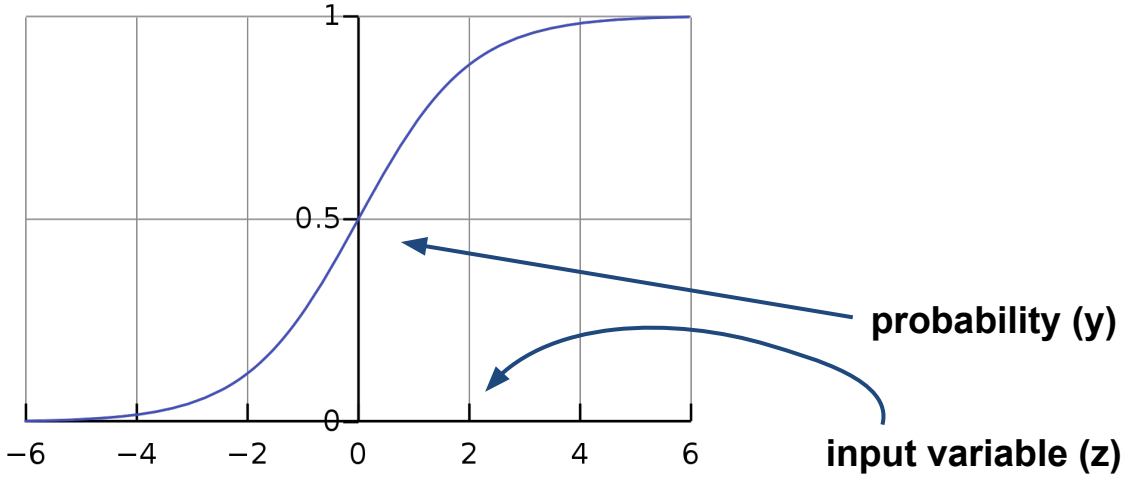
- identity function: a.k.a. **linear activation** function
- returns the value z that comes from the combination of input features and learned weights
- **never used**, except for the output layer in regression problems



Activation functions: logistic function

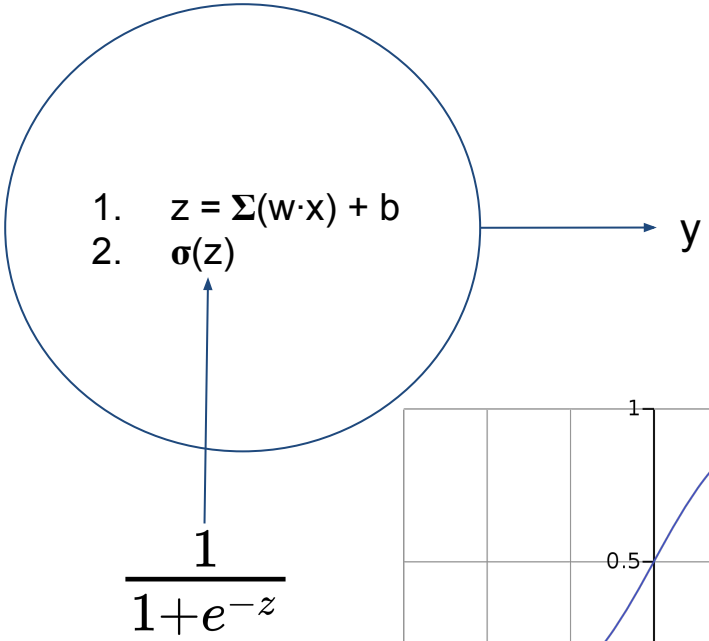


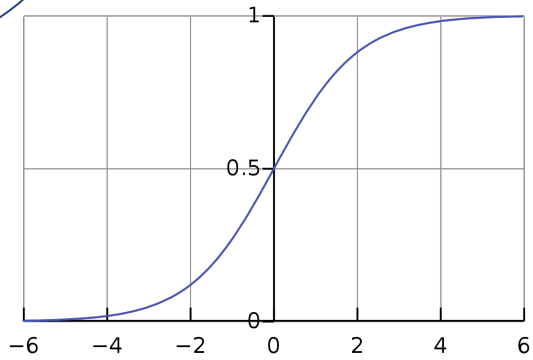
- logistic (**sigmoid**) function
- converts real input in [-∞,+∞] to output in the range [0,1]





Activation functions: logistic function

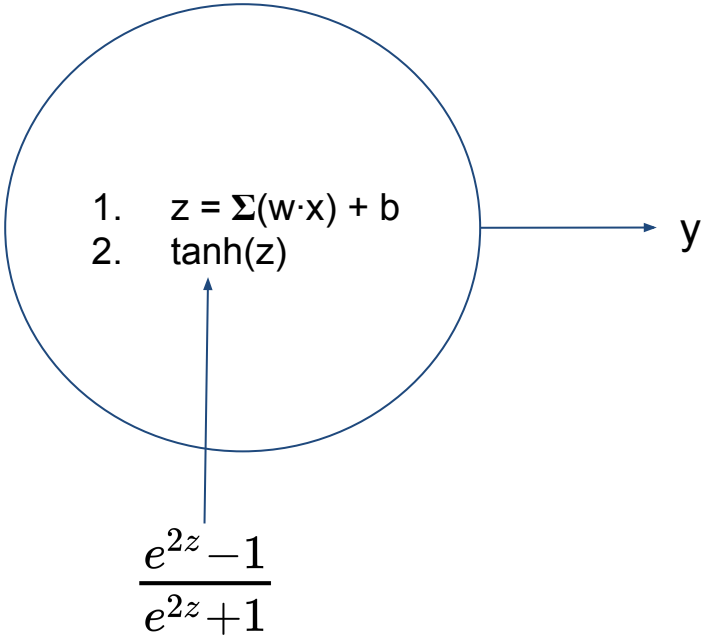




- historically very popular
- now less popular → problems with gradient descent (solution of the model)
- when z is very large or very small derivatives are close to 0 → **slow descent**
- still used for the output layer in binary classification problems (and also for specialised hidden layers/units)

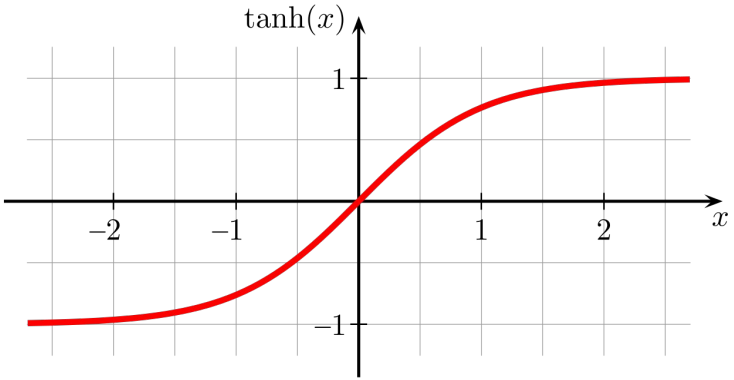


Activation functions: tanh



- hyperbolic tangent function
- rescaling of the logistic function:

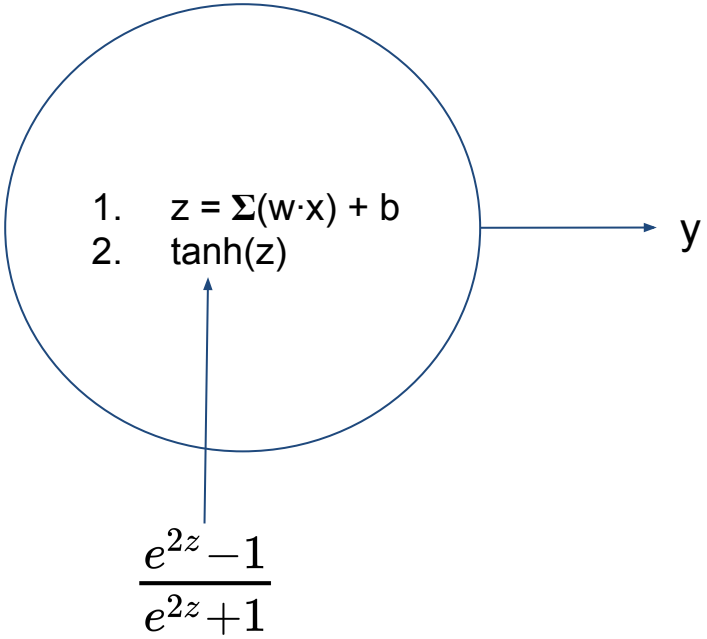
  tanh = 2σ(2z)-1 [proof here]
- output in **[-1,+1]**, **mean 0,** ~ "centering of the data"





From: https://commons.wikimedia.org/wiki/File:Hyperbolic_Tangent.svg

Activation functions: tanh

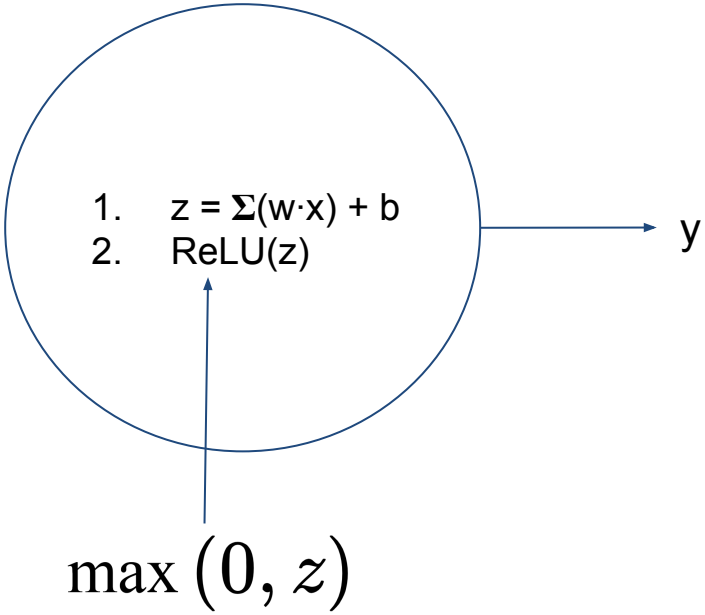


- hyperbolic tangent function
- rescaling of the logistic function:

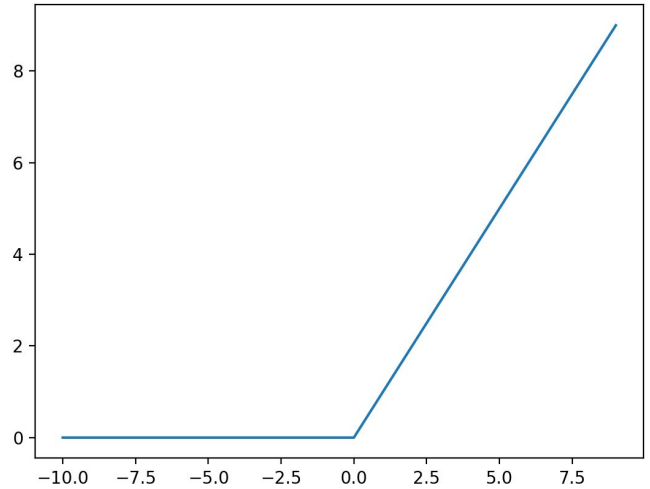
  $\tanh = 2\sigma(2z)-1$ [proof here]
- output in **[-1,+1]**, **mean 0,** ~ "centering of the data"
- more efficient learning in the intermediate hidden layers
- still suffers from similar limitations as $\sigma(z)$ when z is very large or small
- used in specialized layers/units (e.g. RNN)



Activation functions: ReLU



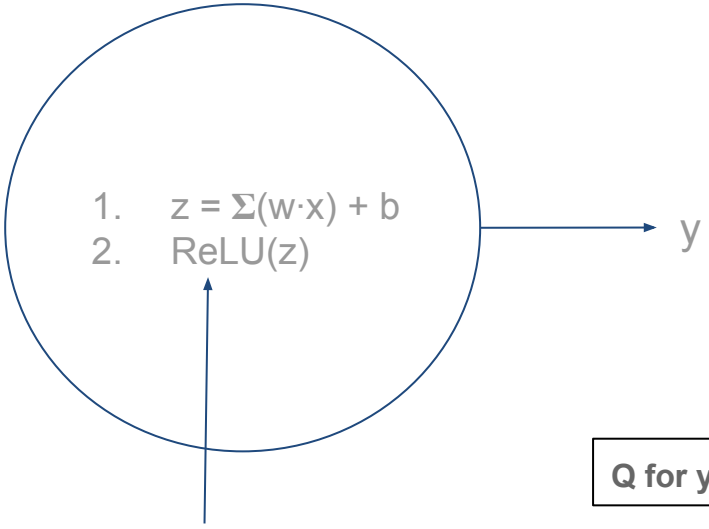
- derivative is 0 for z < 0, 1 for z > 0
- most common activation function (default choice in many cases)
- much faster and efficient learning of DL models





Source: https://machinelearningmastery.com/rectified-linear-activation-function-for-deep-learning-neural-networks/

Activation functions: ReLU



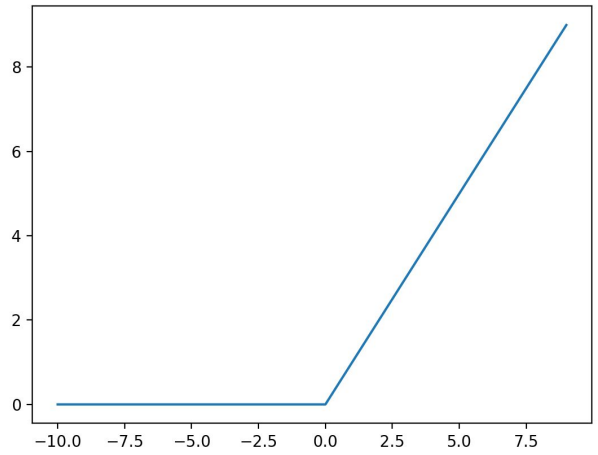
- derivative is 0 for z < 0, 1 for z > 0
- most common activation function (default choice in many cases)
- much faster and efficient learning of DL models

Q for you: which is correct?

$\max(0, z)$

$f(z) = |z|$?

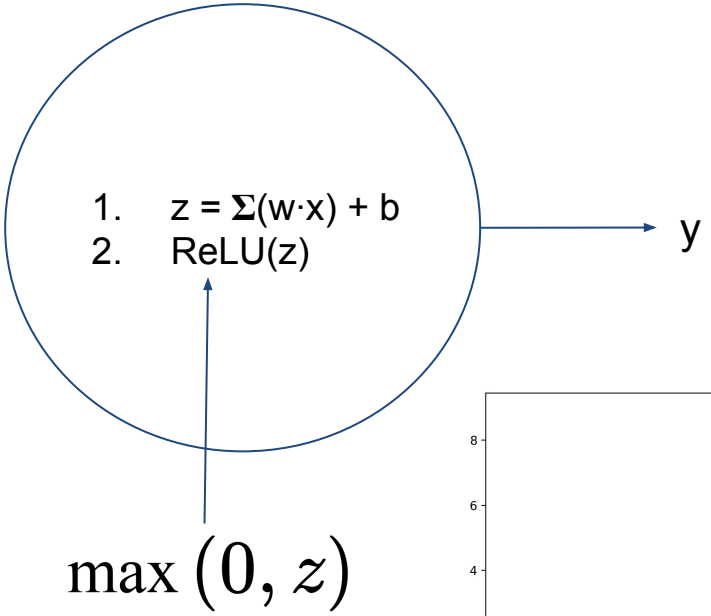
$f(z) = \frac{|z|+z}{2}$?

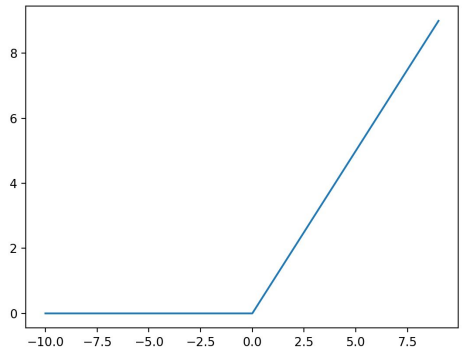




Source: https://machinelearningmastery.com/rectified-linear-activation-function-for-deep-learning-neural-networks/

Activation functions: ReLU





Pros of ReLU activation:

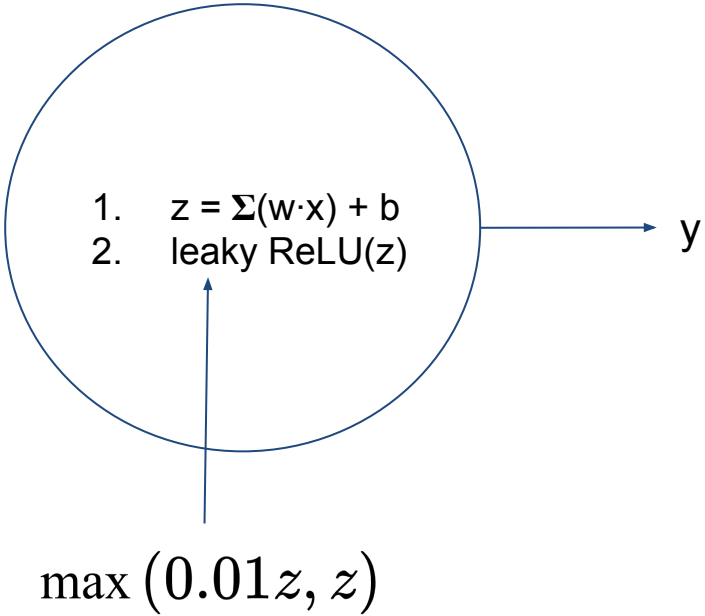
- easy to compute
- sparse representation: many output values will be exactly 0 (unlike sigmoid and tanh, which tends asymptotically to 0)
- reduces vanishing gradients → faster learning (training of multi-layered NNs)

→ ReLU is one of the ingredients that made deep learning possible

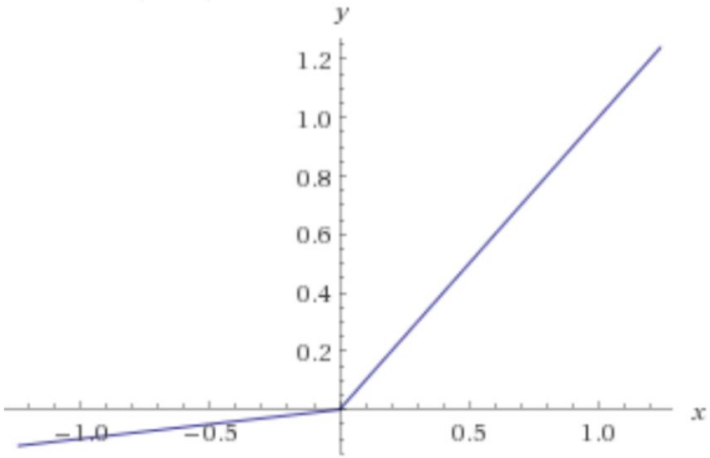


Source: https://machinelearningmastery.com/rectified-linear-activation-function-for-deep-learning-neural-networks/

Activation functions: leaky ReLU



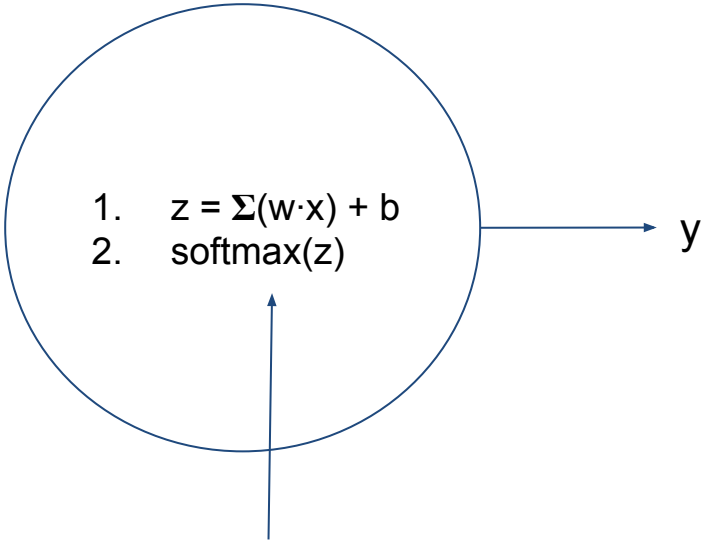
- uses a slight slope for z < 0
- can help when there are too many flat neurons (0 slopes, “dying neurons”), e.g.:
  - large negative bias
  - learning rate is too large





From: http://theprofessionalspoint.blogspot.com/2019/06/dying-relu-causes-and-solutions-leaky.html

Activation functions: softmax



$$\text{softmax}\left(z_i\right) = \frac{exp(z_i)}{\sum_{j=1}^{k} exp(z_j)}$$

- returns a probability distribution over the target classes in a **multiclass classification** problem
- k classes
- negative inputs converted to non-negative values (exponential function)
- each output will be in the interval [0,1]
- same denominator → normalization (sum to 1)

- Softmax is used in the output layer of multinomial classification problems
- Softmax is differentiable → backpropagation for optimization of the weights (parameters of the deep learning model)



Activation functions: why not linear?

- the linear (identity) activation function is never used: why?



Activation functions: why not linear?

- the linear (identity) activation function is never used: why?
- has to do with **function approximation**: NNs (deep learning) are excellent at finding complex non-linear relationships in the data (e.g. between features and target variables)
- with the identity activation function, the intermediate output of each layer will just be a linear combination of the input, and so no matter how many hidden layers you have, the final output **ŷ** will be a **linear combination** of the initial features **X**
- deep learning would then just be a very expensive way of doing linear regression!

layer 1

$$\begin{cases} y_1 = w_1 x + b_1 \\ y_2 = w_2 y_1 + b_2 \end{cases} \rightarrow y_2 = w_2(w_1 x + b_1) + b_2 = (w_2 w_1)x + (w_2 b_1 + b_2)$$

layer 2

w’ b’



Q&A - 1

- Is multiple linear regression (y = mu + beta*x + e) a machine learning method?
- Is multiple linear regression “supervised”?
- What is a fully connected neural network?
- What is a dense layer?
- What are activation functions?

Questions for us?



Logistic regression

From a neural network perspective



Binary classification problems

- the response variable **y** is **qualitative** and takes up one of two values
- binary traits (e.g. cases/controls, resistant/susceptible, true/false, etc.)
- **y** = **label** (a.k.a. target/dependent variable)
- **X** = matrix of **features** (continuous, categorical)



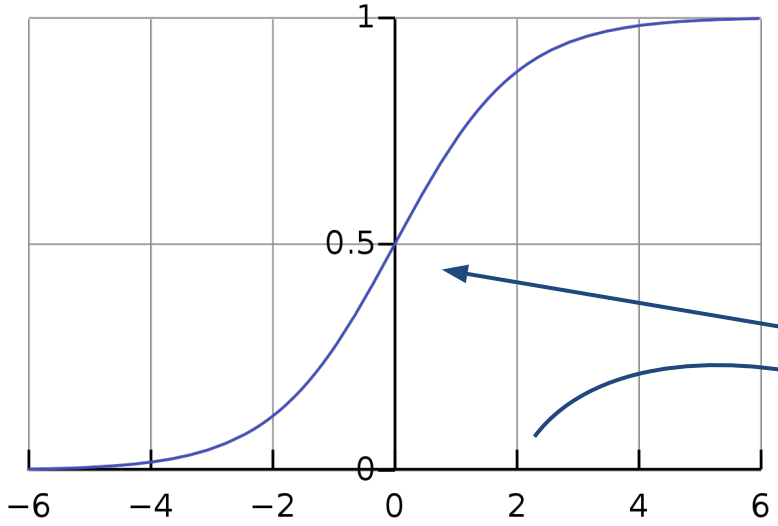
Binary classification problems

- the response variable **y** is **qualitative** and takes up one of two values
- binary traits (e.g. cases/controls, resistant/susceptible, true/false, etc.)
- **y** = **label** (a.k.a. target/dependent variable)
- **X** = matrix of **features** (continuous, categorical)
- we don't model the response directly, rather its **probability**:
  - **P(y=1|X) = f(X)** (probabilities lie in [0,1] (not +/- infinity))

Q1: if you have *n* examples with *m* features X_{nm}, what would $f(X_{nm})$ look like?



The logistic function



P(y=1|z) = f(z)
(probabilities lie in [0,1] (not +/- infinity))

probability (y)

predictor variable (z)

$$\frac{1}{1+e^{-z}} = \frac{1}{1+\frac{1}{e^{z}}} = \frac{e^{z}}{1+e^{z}}$$



Logistic regression

$$P(y=1|x) = \sigma(z) = \frac{e^{\beta_0+\beta_1 x}}{1+e^{\beta_0+\beta_1 x}}$$

we see here the familiar **model coefficients** (from linear regression) to be estimated and then used for predictions (but now they're exponents!)



Logistic regression

$$P(y=1|x) = \sigma(z) = \frac{e^{\beta_0+\beta_1 x}}{1+e^{\beta_0+\beta_1 x}}$$

we see here the familiar **model coefficients** (from linear regression) to be estimated and then used for predictions (but now they're exponents!)

[simplify]

$$\frac{\sigma(z)}{1-\sigma(z)} = e^{\beta_0+\beta_1 x}$$

odds

[simplify further]

$$log\left(\frac{\sigma(z)}{1-\sigma(z)}\right) = logit(\sigma(z)) = \beta_0 + \beta_1 x$$

log(odds): logit



Logistic regression

- the **logit function** (log(odds)) is the **link function** between a linear expression of x and the probabilities of y [(p(y=1|x) = p(x))]
- linear ‘x’ expression (β_0+β_1x) → logit scale (continuous)
- logistic (sigmoid) function: converts values on the logit scale back to probabilities

$$\begin{cases} logit(p(x)) = \beta_0 + \beta_1 x \\ p(x) = \sigma(\beta_0 + \beta_1 x) \end{cases}$$

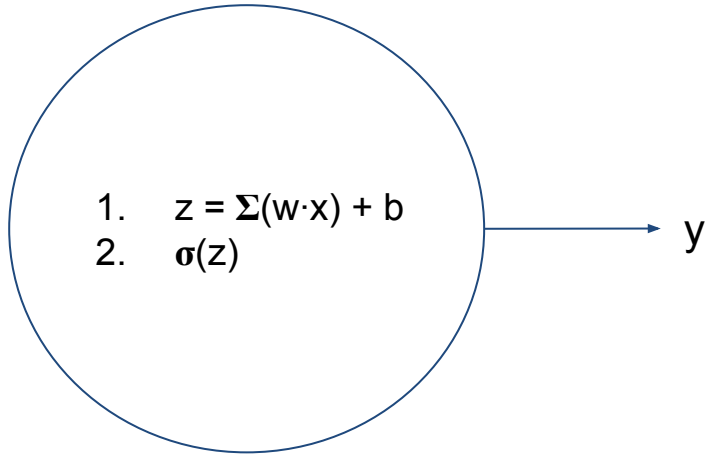
Looks familiar!



Logistic regression

- the **logit function** (log(odds)) is the **link function** between a linear expression of x and the probabilities of y [(p(y=1|x) = p(x))]
- linear ‘x’ expression (β_0+β_1x) → logit scale (continuous)
- logistic (sigmoid) function: converts values on the logit scale back to probabilities

$$\begin{cases} logit(p(x)) = \beta_0 + \beta_1 x \\ p(x) = \sigma(\beta_0 + \beta_1 x) \end{cases}$$



Looks familiar!



Estimating the coefficients

how do we obtain the model coefficients β?



Estimating the coefficients

how do we obtain the model coefficients β?

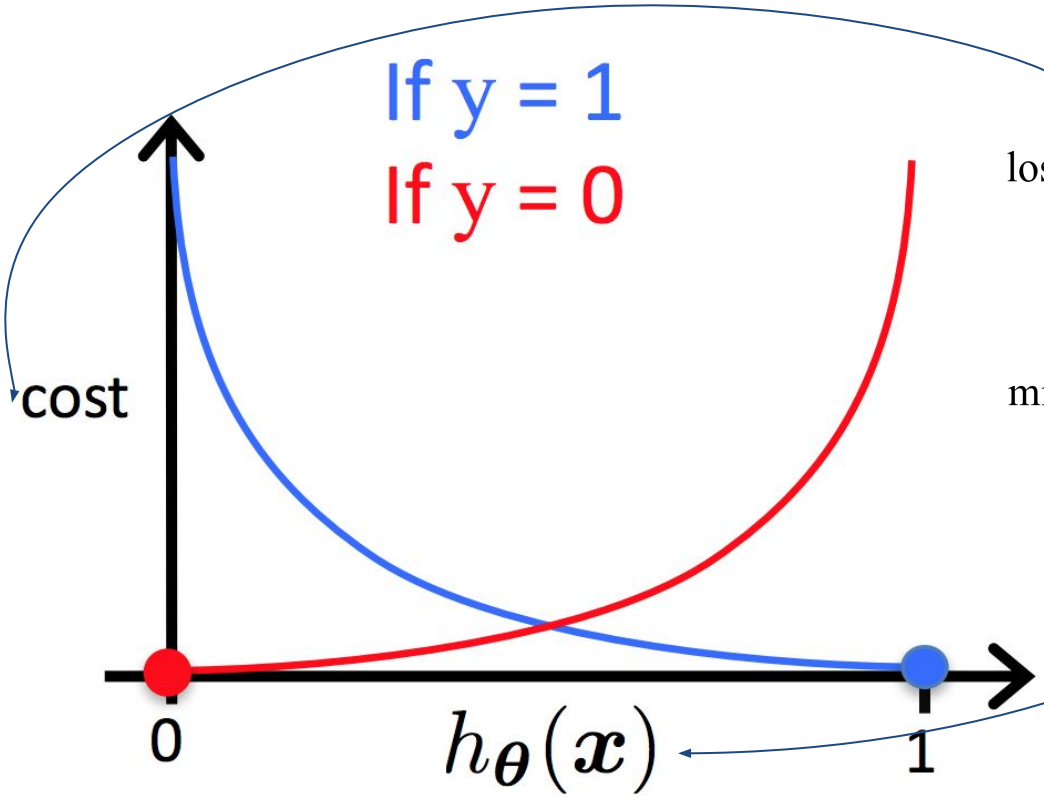
- we need to define a **loss function** and then **minimise** it

observations	predictions
y	$\hat{y} = \sigma(\beta_0 + \beta_1 x)$

difference between observed and predicted values



Loss function for logistic regression



$$\text{loss}\,(\hat{y}, y) = J(\beta)$$

$$J(\beta) = -\left(y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})\right)$$

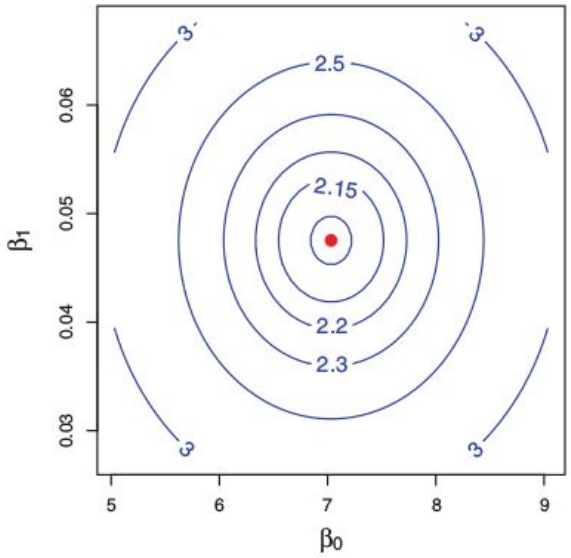
$$\underset{\beta}{\text{minimize}}\, J(\beta)$$

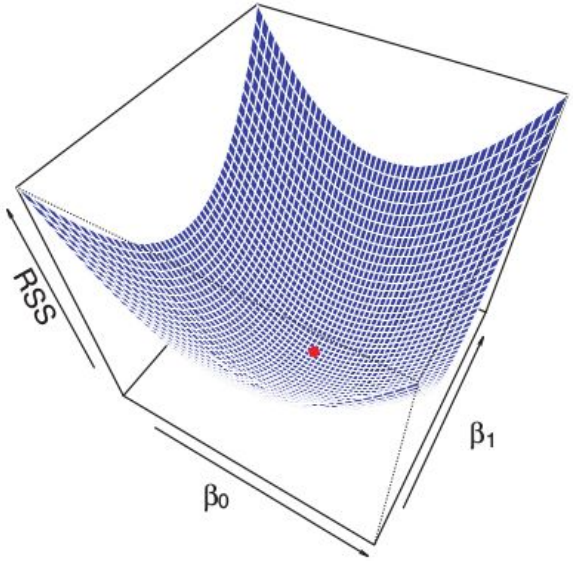
Where is the minimum of this function?

From: https://datascience.stackexchange.com/questions/40982/logistic-regression-cost-function



Minimise the cost function





Multiple logistic regression (e.g. 2 parameters):

$$y = \beta_0 + \beta_1 \cdot x$$

Multiple logistic regression (> 2 parameters):

→ m-dimensional hyperspace

OK: to obtain model coefficients we need to define and then minimise the cost function

How do we minimise the cost function?
Any ideas?



Minimising the cost function

- the defined cost function is convex
- can be minimised by **gradient descent**
- machine learning perspective: gradient descent is a general algorithm to solve models
- alternatively:
    - maximum likelihood
    - non-linear least squares



Cost function: finding the minimum

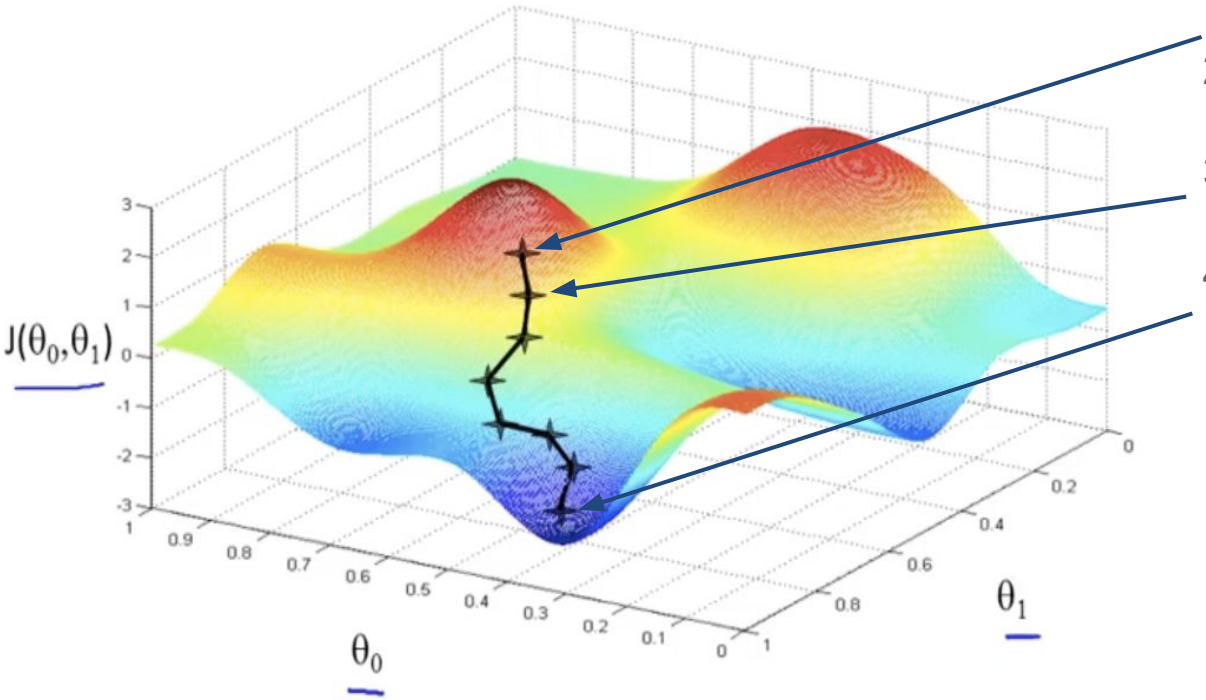
Gradient Descent:

$$\underset{\beta}{\text{minimize}}\, J(\beta)$$

1. Start with initial values for β : (initialisation)
2. Change β in the direction of reducing J(β) : (descent)
3. Stop when the minimum is reached : (minimisation)



Gradient descent

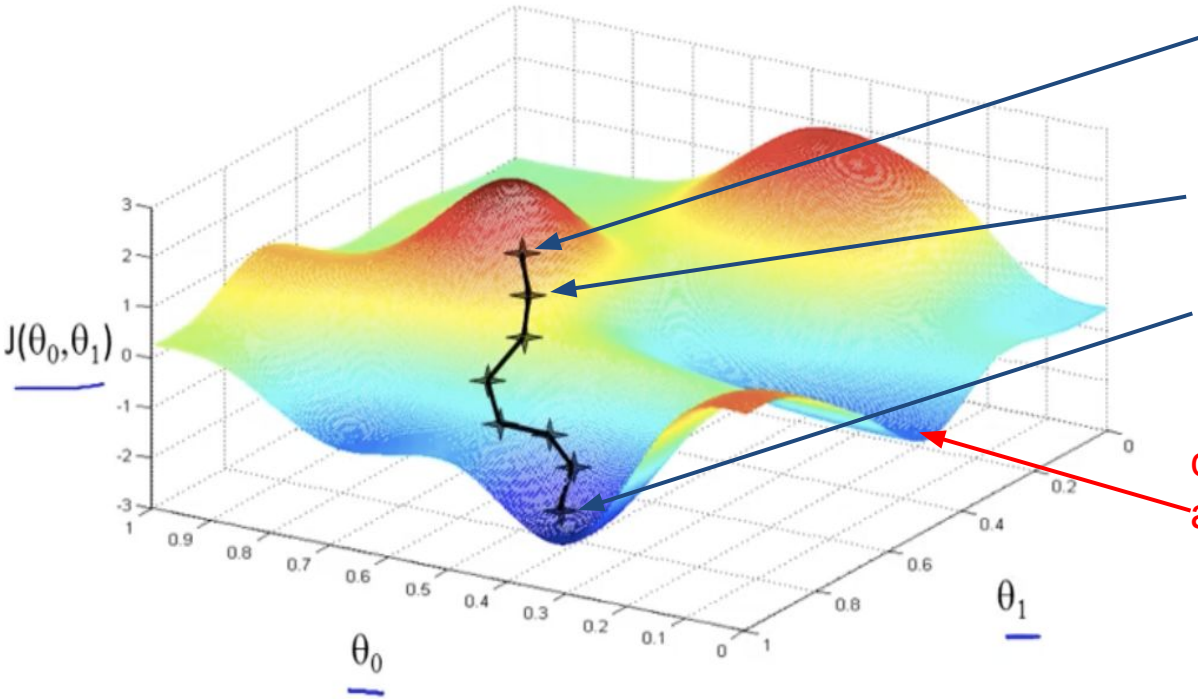


1. starting point (initialisation)
2. find the steepest direction around the starting point
3. take one step in this direction
4. repeat until convergence (local minimum)

Source: Andrew Ng
https://medium.com/@DBCerigo/on-why-gradient-descent-is-even-needed-25160197a635



Gradient descent



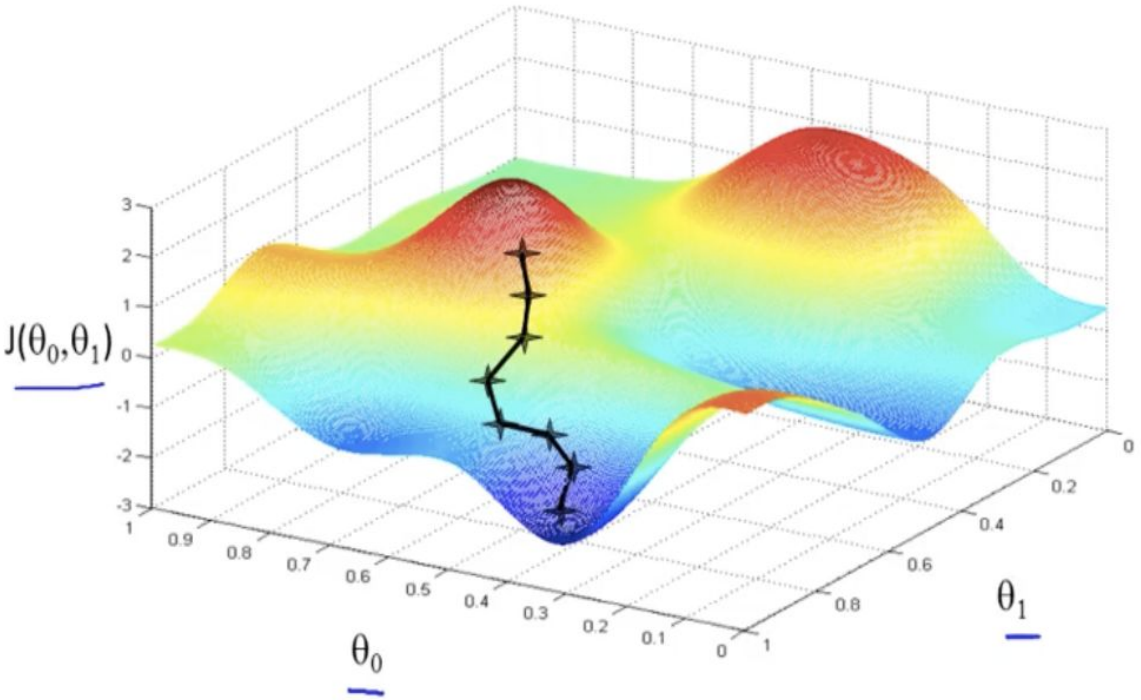
1. starting point (initialisation)
2. find the steepest direction around the starting point
3. take one step in this direction
4. repeat until convergence (local minimum)

changing initialisation may lead to a different local minimum!

Source: Andrew Ng
https://medium.com/@DBCerigo/on-why-gradient-descent-is-even-needed-25160197a635



Gradient descent



How do we find the steepest direction?

Source: Andrew Ng
https://medium.com/@DBCerigo/on-why-gradient-descent-is-even-needed-25160197a635



Gradient descent

new value for βj

old value for βj

$$\beta_{j1} := \beta_{j0} - \alpha \frac{\partial}{\partial \beta_j} J(\beta)$$

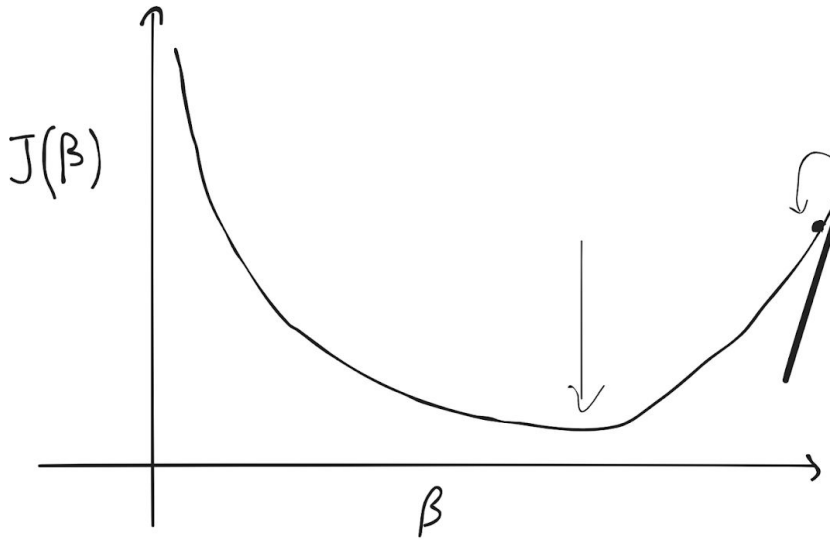
assignment operator

learning rate (size of the steps)

Partial derivative of J(**β**) with respect to $\boldsymbol{\beta}_j$



Gradient descent



- starting point (initial value for β)
- calculate the (partial) derivative in that point
- → positive slope
- **update** the value for β
- repeat

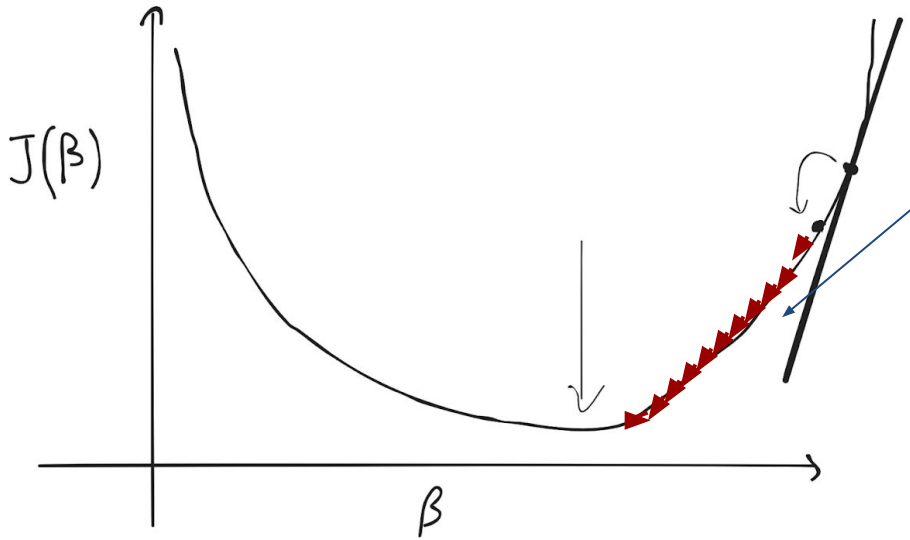
$$\beta_j := \beta_j - \alpha \frac{\partial}{\partial \beta_j} J(\beta)$$

- positive slope → reducing the value of β (and the other way around)

made with ZITEBOARD



Gradient descent

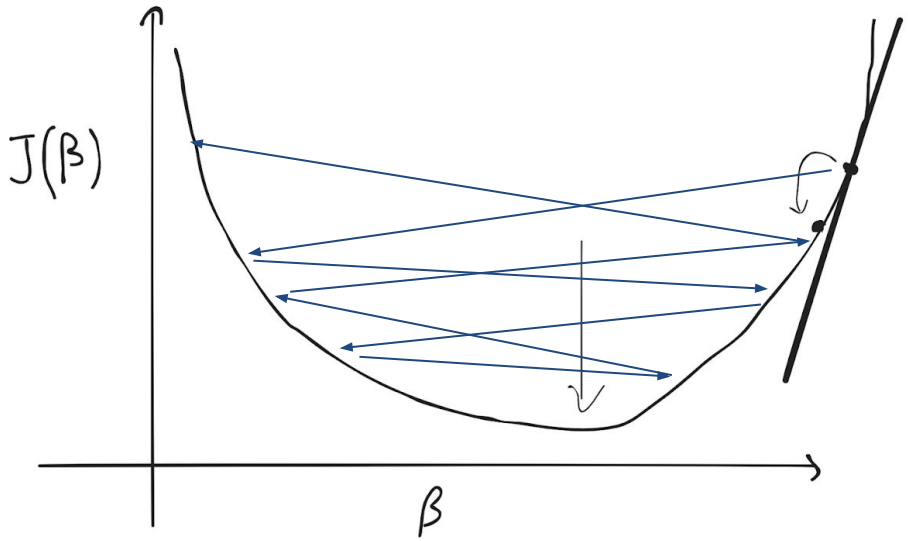


- α controls the size of the updating step
- small $\alpha \rightarrow$ slow descent

$$\beta_j := \beta_j - \alpha \frac{\partial}{\partial \beta_j} J(\beta)$$

made with ZITEBOARD

Gradient descent



- α controls the size of the updating step
- large $\alpha \rightarrow$ overshooting: failure to converge

$$\beta_j := \beta_j - \alpha \frac{\partial}{\partial \beta_j} J(\beta)$$

made with ZITEBOARD

Cost function: finding the minimum?

Why algorithmic solution (series of steps) and not analytical solution?

- not always a closed form solution is available
- matrix derivatives are quite difficult and complex to calculate
- computational complexity (matrix inversion ~ $O(N^3)$); stepwise algorithm is more efficient
- algorithms are easier to program than analytical calculations
- etc.



Cost function: finding the minimum?

Why algorithmic solution (series of steps) and not analytical solution?

- not always a closed form solution is available
- matrix derivatives are quite difficult and complex to calculate
- computational complexity (matrix inversion ~ $O(N^3)$); stepwise algorithm is more efficient
- algorithms are easier to program than analytical calculations
- etc.

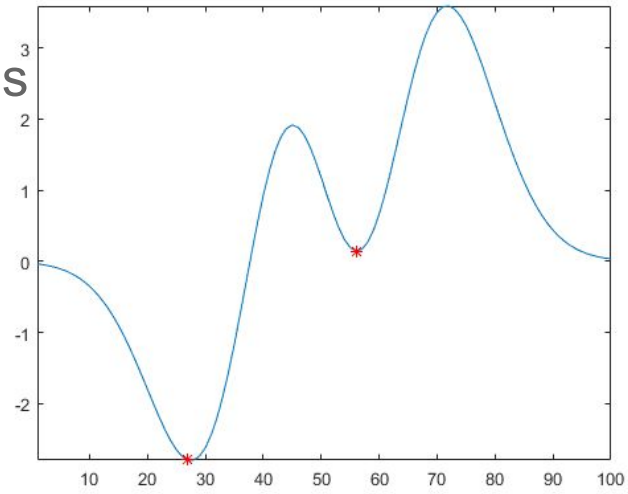
Mind you: in the algorithmic solution (gradient descent & co.) you will still need to take derivatives (but partial derivatives at specific data points)



Gradient descent - recap

- general method to **solve machine learning models** (e.g. multiple linear regression)
- optimise (minimise) the cost function → **optimiser**
- importance of the **learning rate**

- local minimum → **momentum** (intuition: the moving average of previous updates is weighed in into the calculations)
- **initialization** values





Logistic regression - recap

1) $z = w \cdot x + b$

2) $\hat{y} = \sigma(z)$

3) $J(w) = -\left(y \cdot log(\hat{y}) + (1 - y) \cdot log(1 - \hat{y})\right)$

introduce **w** (weight) as parameter (+ b): NN notation



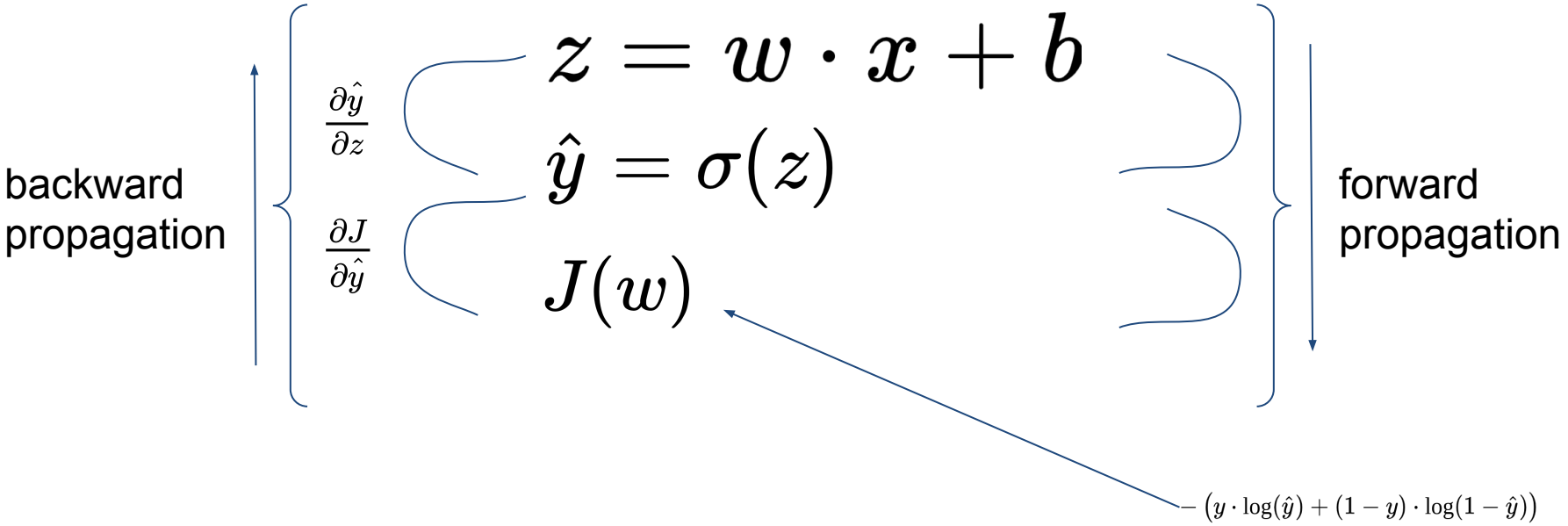
Logistic regression NN: forward propagation

1) $z = w \cdot x + b$
2) $\hat{y} = \sigma(z)$
3) $J(w)$

forward propagation



Logistic regression: backpropagation





Logistic regression: backpropagation

backward propagation

$$\frac{\partial \hat{y}}{\partial z}$$

$$\frac{\partial J}{\partial \hat{y}}$$

$$z = w \cdot x + b$$

$$\hat{y} = \sigma(z)$$

$$J(w)$$

forward propagation

$$-\left(y \cdot \log\left(\hat{y}\right) + (1-y) \cdot \log\left(1-y\right)\right)$$

$$\frac{\partial J}{\partial z} = \frac{\partial J}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z}$$

chain rule



Logistic regression: backpropagation

$$\frac{\partial J}{\partial z} = \frac{\partial J}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z}$$

our objective!

$$\frac{\partial J}{\partial w_1} = \frac{\partial J}{\partial z} \cdot x_1$$

$$\frac{\partial J}{\partial w_2} = \frac{\partial J}{\partial z} \cdot x_2$$

$$\frac{\partial J}{\partial b} = \frac{\partial J}{\partial z}$$

$$w_1 := w_1 - \alpha \frac{\partial J}{\partial w_1}$$

$$w_2 := w_2 - \alpha \frac{\partial J}{\partial w_2}$$

$$b := b - \alpha \frac{\partial J}{\partial b}$$

updating step



NN for logistic regression - epochs

backward propagation

$$\frac{\partial \hat{y}}{\partial z}$$

$$\frac{\partial J}{\partial \hat{y}}$$

$$z = w \cdot x + b$$

$$\hat{y} = \sigma(z)$$

$$J(w)$$

forward propagation

forward + backward propagation = epoch



Take away message

- **back propagation** is a way (algorithm) to calculate partial derivatives (of the cost function with respect to the parameters) easily and efficiently
- partial derivatives are then used to **update** the values of the **parameters**
- in this way **gradient descent** can work to **minimise** the **cost function** and **estimate** the best values for the **parameters**
- this is very important to efficiently learn the weights (parameters) of deep neural networks



Q&A - 2

- What does the cost function measure?
- What happens during forward propagation?
- What happens during backward propagation?
- What is an epoch?

Questions for us?



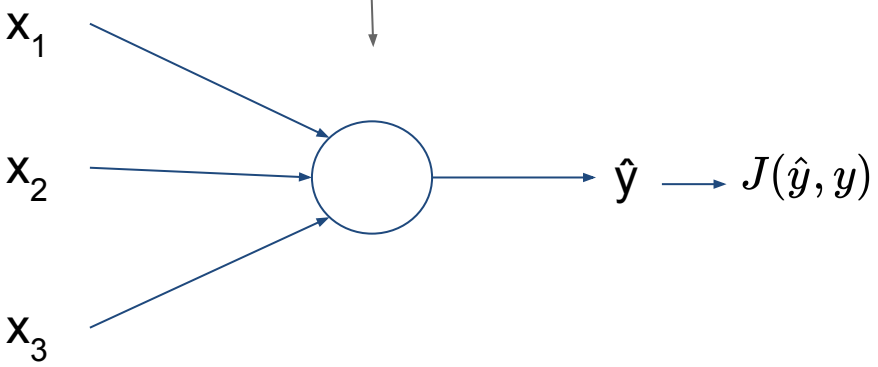
From logistic regression to neural networks

Binary classification



Logistic regression as a neural network

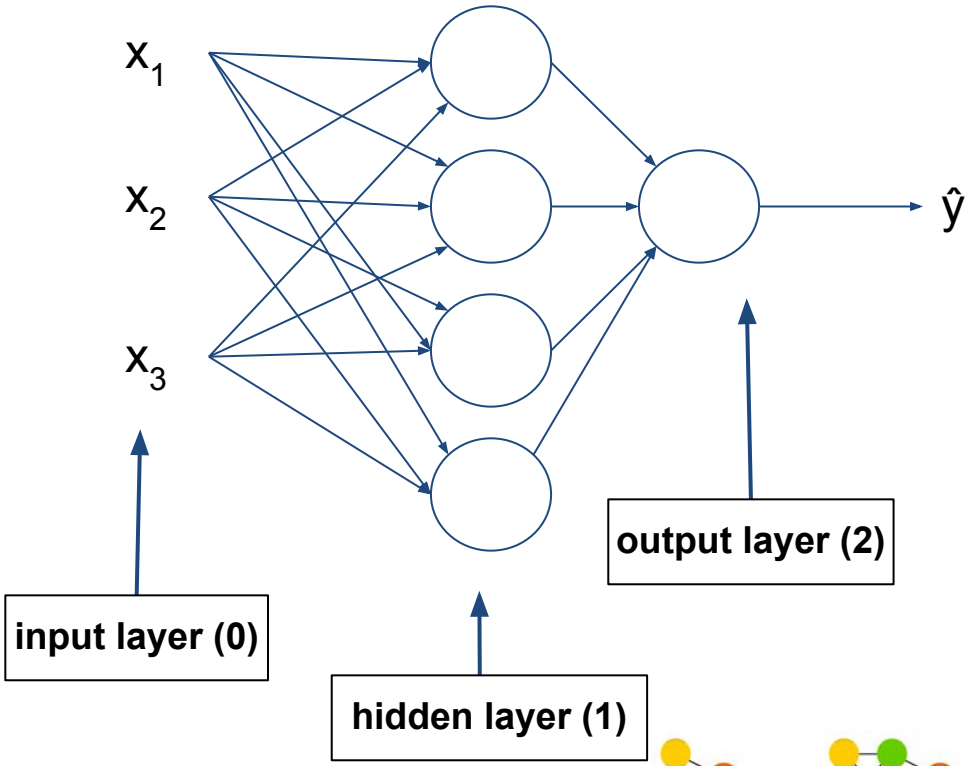
$$z = w \cdot x + b \longrightarrow \hat{y} = \sigma(z)$$



- input variables (features): x1, x2, x3
- calculations in the unit/neuron
- forward and back propagation
- this is just one single neuron!



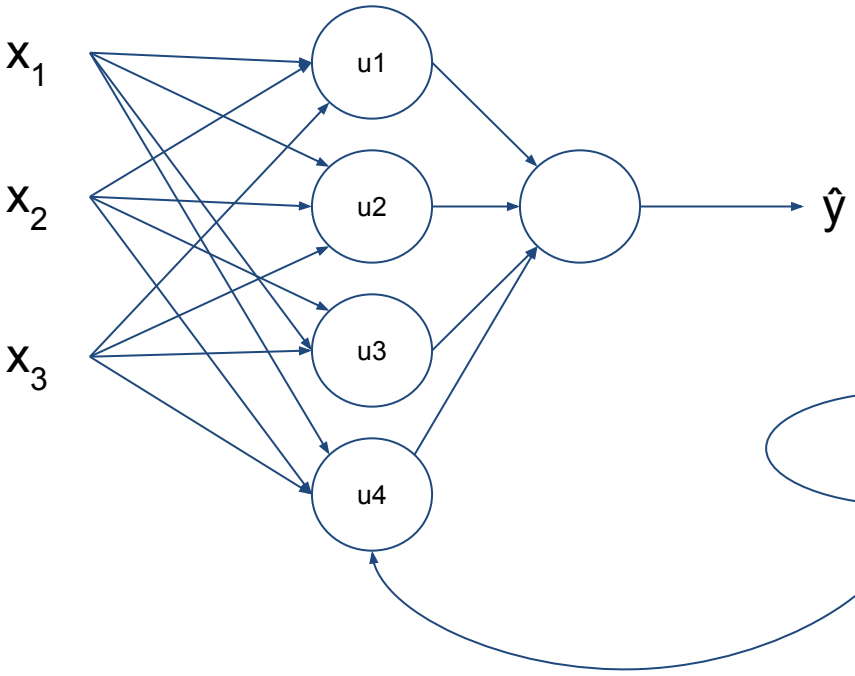
Logistic regression as a neural network



- <u>two layers</u>:
  - **1 hidden layer, 4 nodes**
  - **1 output layer, 1 node**
- logistic regression is performed in each node
- each node in the hidden layer receives all input variables
- the node in the output layer receives all outputs (**activations**) from the hidden layer nodes

Q: what differs between one logistic regression model and the other?

Logistic regression as a neural network



n observations, **m** features, **u** units

$$\mathbf{Z}^{[1]}_{(n,u)} = \mathbf{X}_{(n,m)} \cdot \mathbf{W'}^{[1]}_{(m,u)} + \mathbf{b}^{[1]}_{(1,u)}$$

$$\mathbf{A}^{[1]}_{(n,u)} = \sigma(\mathbf{Z}^{[1]})$$

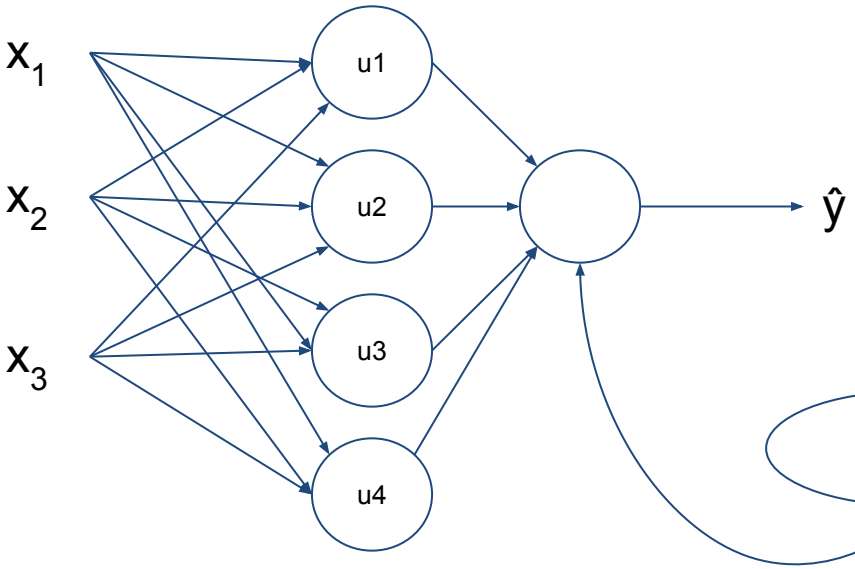
original input data matrix **X**

A for activation (output of the hidden layer)

What do we have in matrix A?



Logistic regression as a neural network



n observations, **m** features, **u** units

$$\mathbf{Z}^{[1]}_{(n,u)} = \mathbf{X}_{(n,m)} \cdot \mathbf{W'}^{[1]}_{(m,u)} + \mathbf{b}^{[1]}_{(1,u)}$$

$$\mathbf{A}^{[1]}_{(n,u)} = \sigma(\mathbf{Z}^{[1]})$$

$$\mathbf{Z}^{[2]}_{(n,1)} = \mathbf{A}^{[1]}_{(n,u)} \cdot \mathbf{W'}^{[2]}_{(u,1)} + \mathbf{b}^{[2]}_{(1,1)}$$

$$\hat{\mathbf{y}}_{(n,1)} = \sigma(\mathbf{Z}^{[2]})$$

What do we have in vector ŷ?



Take away messages

- You can build a neural network (NN) for binary classification
- NNs are logistic regression repeated several times! → n. of nodes/units, n. of layers
- probably a bit of an overkill to use NNs in place of a simple logistic regression model → used for illustration
- however, when you have many observations and many features (big data), NN will do the job

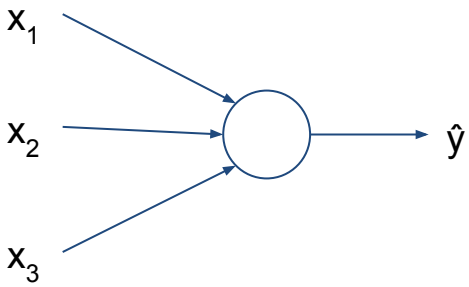


Let’s go deep

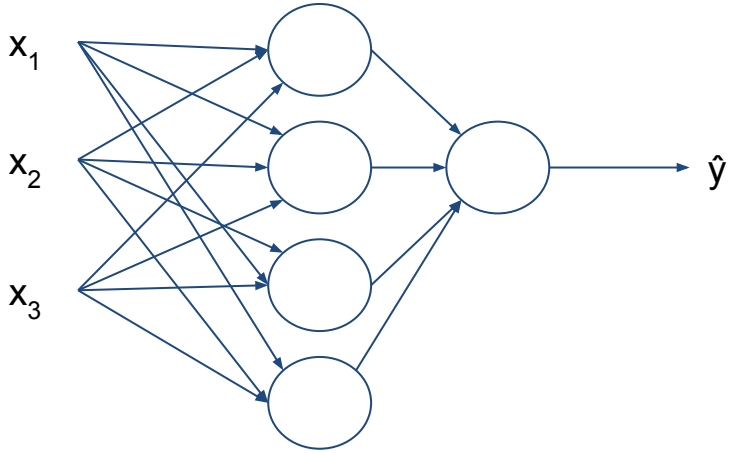
From NNs to deep learning



It’s a matter of layers



logistic regression
(1 layer)

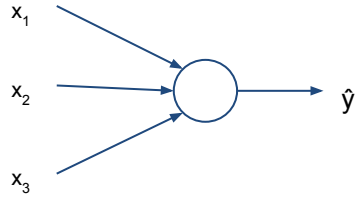


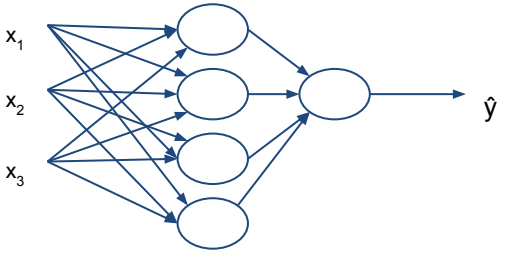
shallow NN
(2 layers)



It’s a matter of layers

- we had a (cursory) look at the calculations involved in neural networks models
- Lots of details, but it’s **not a black box**!

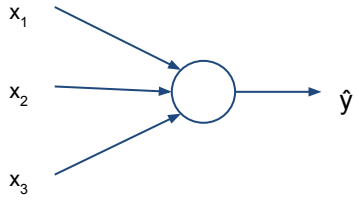


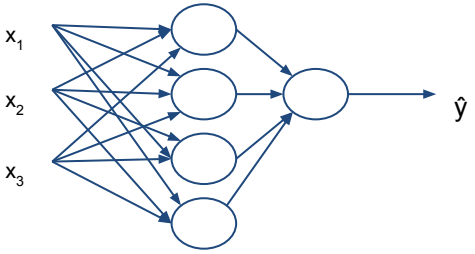




It’s a matter of layers

- we had a (cursory) look at the calculations involved in neural networks models
- Lots of details, but it’s **not a black box**!
- however, when you go deep (more layers), the magic gets back in the play!

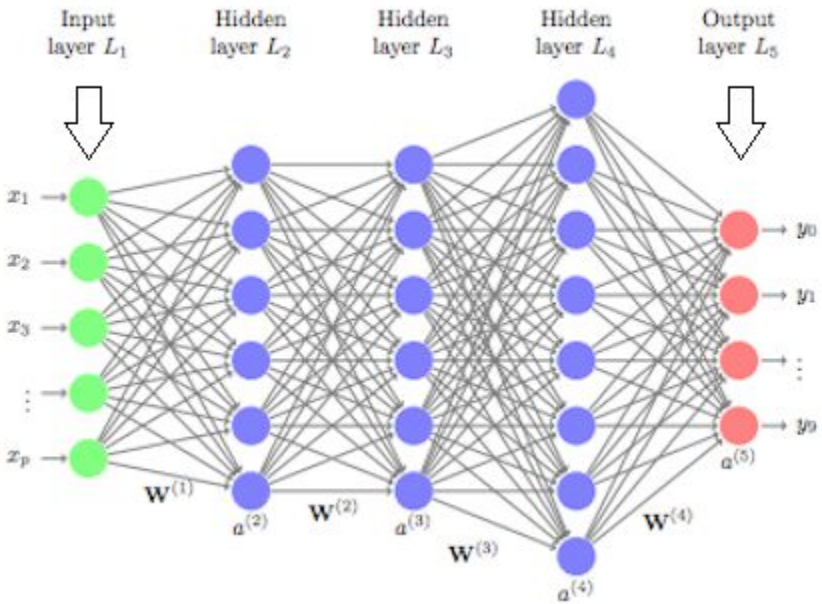








It’s a matter of layers

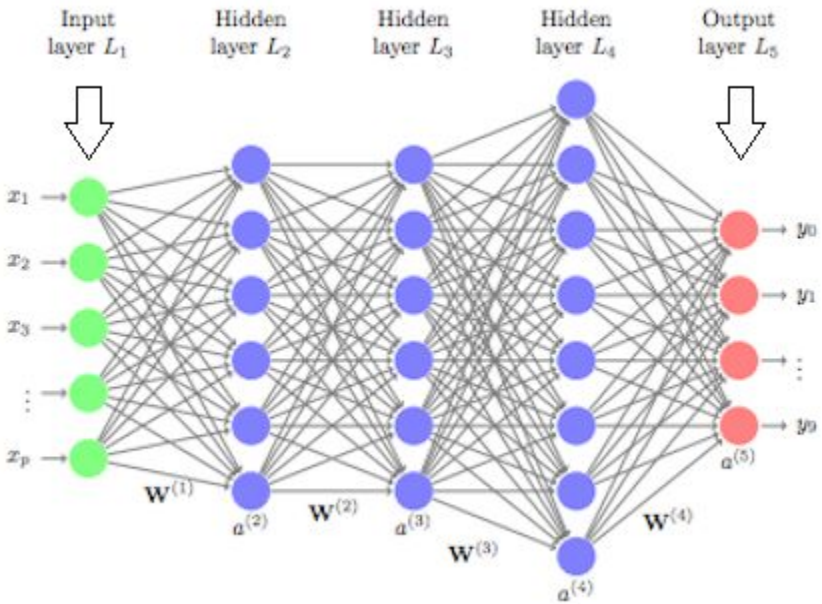


Source: University of Cincinnati

- 4 layers (“deep” NN)
- n. of layers in a deep learning model: hyperparameter to tune (one of many)



Layers matter!

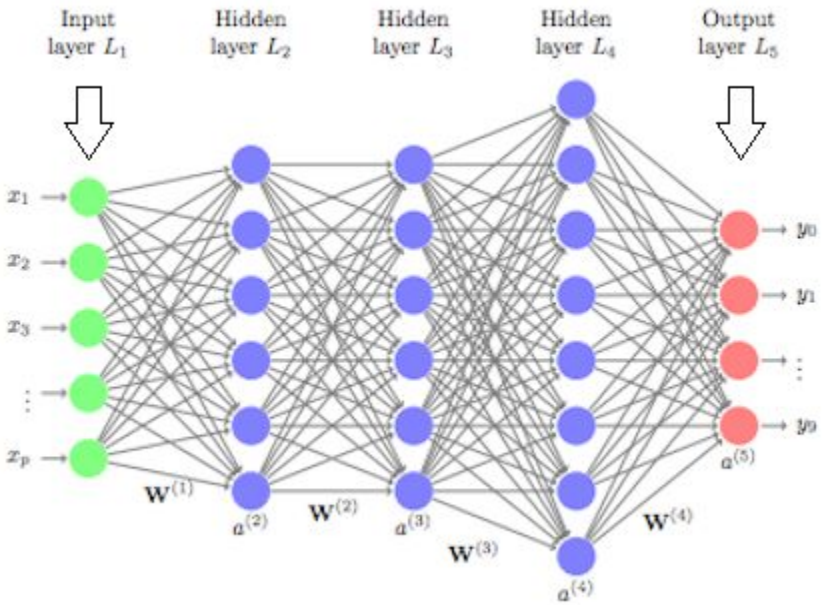


Source: University of Cincinnati

- 4 layers (“deep” NN)
- n. of layers in a deep learning model: hyperparameter to tune (one of many)
- research in the AI/machine learning communities has shown that there are functions that deep NN can learn which can not be learnt by shallower models



Forward propagation



Source: University of Cincinnati

- L: n. of layers (l in 1 to L)
- X: matrix of input features → $A^{[0]}$

$$\begin{cases} \mathbf{Z}^{[l]} = \mathbf{A}^{[l-1]} \cdot \mathbf{W}'^{[l]} + \mathbf{b}^{[l]} \\ \mathbf{A}^{[l]} = g^{[l]}(\mathbf{Z}^{[l]}) \end{cases}$$

- for each layer
- iterate over n. of layers (unavoidable for loop)
  - N. nodes → memory
  - N. layers → CPU

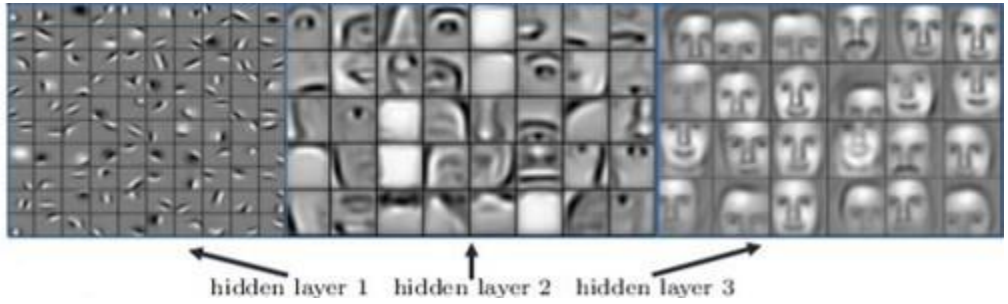


Compositional representation

- why do deep NNs work better?
- **each layer** focuses on **one representation** of the data
- representations are then **combined** to get the final result → **compositional representation**

Simplified example from face recognition:

- Layer 1 → edges
- Layer 2 → pieces of faces
- Layer 3 → contours



Source: https://medium.com/@fenjiro/face-id-deep-learning-for-face-recognition-324b50d916d1



Compositional representation

- works also with other types of data
- e.g. speech recognition: i) high/low soundwaves; ii) combinations of soundwaves into phonemes; iii) combination of phonemes into words; iv) from words to sentences
- relatively simple functions of the input data in the first layers → **progressively more complex functions** of the data in the later layers



Depth vs width

- deep learning works by stacking together multiple (many) hidden layers with relatively few nodes
- alternatively, one could use a shallow but very wide (many nodes) neural network
- **depth is more efficient than width**: shallow NN require exponentially more hidden units (nodes) compared to deep NN:
    - deep NN $\rightarrow$ O(log(n))
    - wide NN $\rightarrow$ $O(2^n)$



Hyperparameters

- we saw that there are many ingredients that make up a deep learning model (and many still yet to come) → deep learning has **many hyperparameters**:
  - learning rate α
  - n. of hidden layers
  - n. of nodes (total, each layer)
  - activation function
  - and many more (mini-batch size, NN architecture, regularization etc.)
- to be fine-tuned (→ cross-validation)

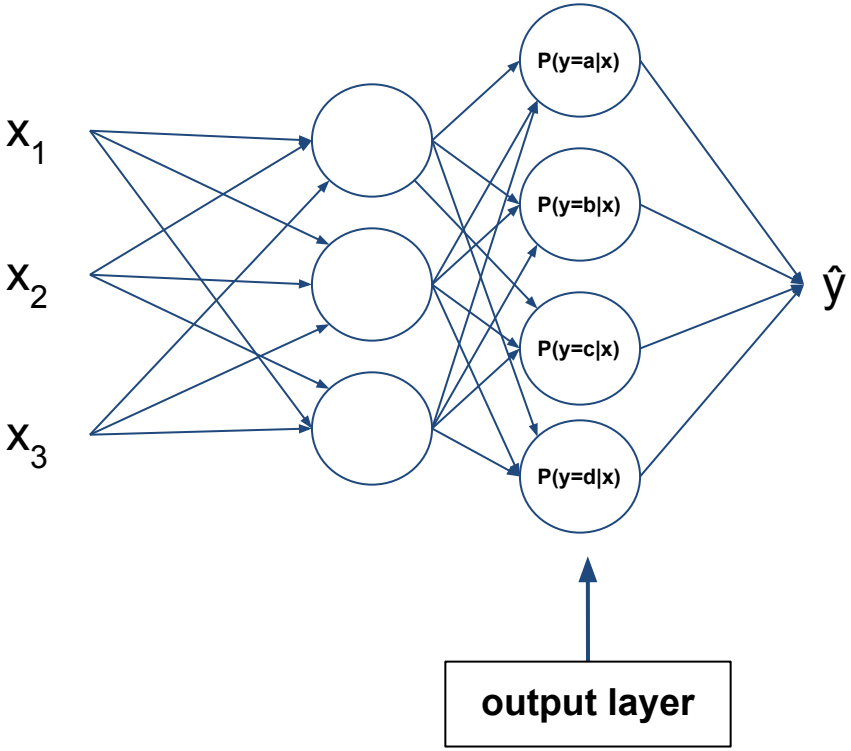


Multiclass classification with neural networks

Softmax regression



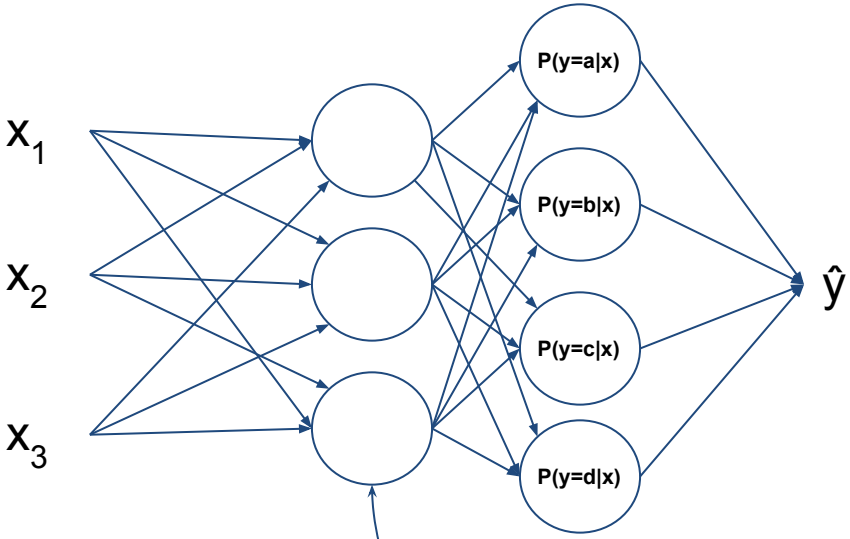
Multiclass classification



- the **output layer** has now **4 nodes** (instead of just one as in binary classification)
- the **activation** function is now the **softmax function**
- this is why the output layer is also called the **softmax layer**, and multiclass classification is called **softmax regression**



Multiclass classification



n observations, **m** features, **u** units, **c** classes

$$\mathbf{Z}^{[1]}_{(n,u)} = \mathbf{X}_{(n,m)} \cdot \mathbf{W}'^{[1]}_{(m,u)} + \mathbf{b}^{[1]}_{(1,u)}$$

$$\mathbf{A}^{[1]}_{(n,u)} = \mathrm{ReLU}\left(\mathbf{Z}^{[1]}\right)$$

original input data matrix **X**

A for activation (output of the hidden layer)

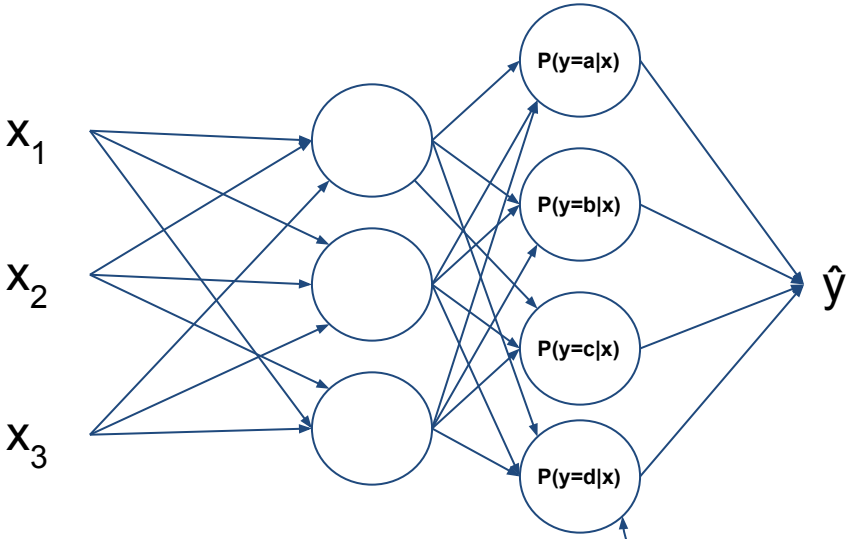
layer 1

softmax layer

What do we have in matrix A?



Multiclass classification



n observations, **m** features, **u** units, **c** classes

$$\mathbf{Z}^{[1]}_{(n,u)} = \mathbf{X}_{(n,m)} \cdot \mathbf{W}'^{[1]}_{(m,u)} + \mathbf{b}^{[1]}_{(1,u)}$$

$$\mathbf{A}^{[1]}_{(n,u)} = \mathrm{ReLU}\left(\mathbf{Z}^{[1]}\right)$$

layer 2

$$\mathbf{Z}^{[2]}_{(n,c)} = \mathbf{A}^{[1]}_{(n,u)} \cdot \mathbf{W}'^{[2]}_{(u,c)} + \mathbf{b}^{[2]}_{(1,c)}$$

$$\hat{\mathbf{y}}_{(n,c)} = \mathrm{softmax}\left(\mathbf{Z}^{[2]}\right)$$

softmax layer

softmax activation

c, classes: ŷ is now a matrix (4 probabilities per sample)



Loss function for softmax regression

$$L(\hat{y}, y) = -\left(y \cdot \log\left(\hat{y}\right) + (1-y) \cdot \log\left(1-\hat{y}\right)\right)$$

loss function for logistic regression

$$L(\hat{y}, y) = -\sum_{j=1}^{c} y_j \cdot \log\left(\hat{y}_j\right)$$

loss function for softmax regression

these are actually vectors of length c (number of classes)!

- generalization of the loss function for logistic regression over c classes
- you then sum up the loss for each sample and divide by the number of samples → cost function for the entire dataset



Recap

- we can represent logistic and softmax regression as neural networks models
- when we add layers and units we **repeat several times the regression model** with a twist given by the ReLU activation
- these multiple regression models all use the same input data, but **are all different** because they learn different weights (parameters)
- as we progress along the layers, we get farther and farther from the original input data → we learn new (more abstract) representations of the data
- when we “go deep” we move away from the linear decision boundaries of logistic (and softmax regression) and learn complex non-linear functions of the data

